



ByteBites: Smart Campus Dining Service Platform

Group Number: 7043

Group Members: Guo Xingchen & Dai Yangjie

Content Table

Content Table	2
Motivation	3
Aim	4
User Stories	4
Milestone 3 Plan - Prototyping (1 Jul - 28 Jul)	5
Features (Overall)	6
Features Implementation	9
Software structure diagram	9
Registration & Login Features	9
1. Registration	9
2. Login	11
3. Log out	13
Student User Features	16
1. Bottom Navigation Bar Implementation:	16
2. Profile picture change:	17
3. Canteen Menu:	18
4. Canteen & dishes review:	20
5. Location Functionality:	24
6. Canteen Sorting and Filtering:	25
7. Vendor Menu Updates Trigger Red Dot Notifications:	27
8. Announcement Bar:	29
9. Favorites Feature:	30
10. Filter Favorited Canteens:	31
11. Canteen Queue Estimation (an imperfect function now):	33
12. Language Preferences:	35
13. Select Preferences:	36
14. Explore Users with Similar Preferences:	39
15. Function Interaction:	41
15.1 Improvement of fun interaction	44
Vendor User features	47
1. Vendor Restaurant Management:	47
2. Reply to Students' Reviews:	49
3. Dashboard Analysis	50
4. Quick Access	51
5. Vendor Reviews	53
6. Vendor Announcements	56
Backend (Spring boot -> java)	59
1. Overview	59
2. Authentication and Authorization Integration	59
3. Canteen Data Management Integration	59
4. Dish Review System Integration	60
5. Menu Management System Integration	61

6. User Profile System Integration	62
7. Frontend-Backend Communication Details	62
8. Security Measures	62
9. Cross-Platform Compatibility	63
10. Configuration	63
Team Coordination & Quality Assurance	64
Testing Process	65
Introduction	65
Use of Trello	66
Software Engineering Practices	67
Tech Stack	68
Future improvements	69
Remote testing	70

Motivation

1. Time Management Challenges: With heavy academic workloads, students need to make quick dining decisions. However, the lack of efficient access to relevant information leads to wasted time when choosing meals.

2. Impact on Healthy Eating Habits: Due to uncertainty about queue lengths and food availability, some students may miss optimal meal times or even skip meals, which could negatively affect their health in the long run.

3. Scattered and Hard-to-Access Information: Existing school platforms do not integrate key dining-related information, such as queue conditions and food reviews. As a result, students must manually compare prices, waiting times, and food quality, making decision-making inefficient.

4. Unmet Diverse Dietary Needs: Students with specific dietary preferences (e.g., halal, vegetarian, gluten-free) lack a convenient way to filter and find suitable food options.

5. Unclear Seating Availability and Dining Environment: Many students not only care about food but also about the dining environment and seat availability. However, there is currently no way to access this information in real time.

6. Low Price Transparency: Since food prices vary across different stalls, students lack a tool for quick price comparisons, which may result in additional expenses or suboptimal choices.

7. Uncertain Canteen Operating Hours and Food Availability: Some food stalls have irregular schedules, and popular dishes may sell out quickly. Without real-time updates, students may struggle to plan their meals efficiently.

Aim

To help students plan their meals more efficiently, we aim to develop a smart application that integrates real-time data from various canteens, including queue durations, menu information, and peak hours. This will enable students to better allocate their time, optimize dining decisions, and improve overall mealtime efficiency, making their diet more organized and healthy. Additionally, our solution will help operators receive quicker feedback on their offerings, allowing them to refine and improve their food quality. Through this intelligent decision-making tool, we strive to enhance the overall campus dining experience.

User Stories

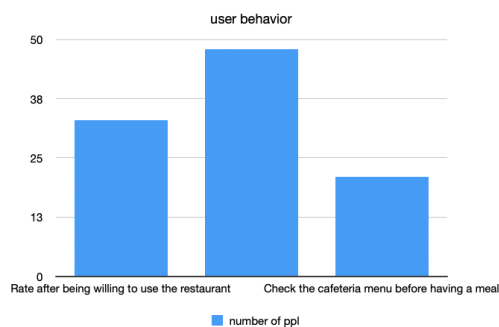
- As an NUS student looking for a quick meal, I want to check real-time queue times so I can avoid long waiting periods.
- As a budget-conscious student, I want to compare food prices across different canteens to make cost-effective dining choices.
- As a health-conscious student, I want personalized meal recommendations based on my dietary preferences.
- As a new student unfamiliar with the canteen options, I want to read reviews and ratings to decide where to eat.

- As a student who wants to contribute to the community, I want to leave reviews, ratings, and queue-time updates to help others.

Milestone 3 Plan - Prototyping (1 Jul - 28 Jul)

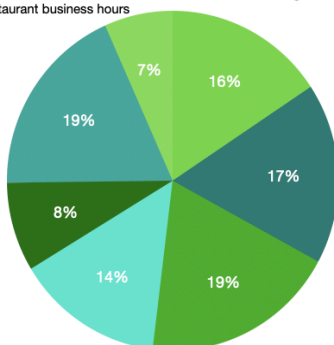
Before starting the project, we make a survey around the school. Based on the survey data related to the user's favorite UI style and behavior, we gathered key insights for software development. The most popular features from the feedback, real-time queue tracking, and label screening of dishes have been prioritized for implementation.

It is precisely because of this investigation that we have become clearer about the software features we want to create.



	number of ppl
Rate after being willing to use the restaurant	33
Check the queue situation in the cafeteria proactively before having a meal	48
Check the cafeteria menu before having a meal	21

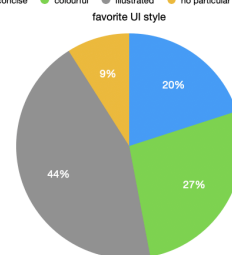
- Check menus
- feedback
- Collect favorite restaurants/dishes
- Restaurant business hours
- real time queue
- Recommend meals as required
- Label screening of dishes



useful feature

	number of ppl
Check menus	61
real time queue	67
feedback	74
Recommend meals as required	56
Collect favorite restaurants/dishes	33
Label screening of dishes	72
Restaurant business hours	26

- concise
- colourful
- illustrated
- no particular preference



表格 2

	concise	colourful	illustrated	no particular preference
favorite UI style	20	27	44	9

Project Setup and Foundation

1. GitHub Repository and Project Creation

Established project repository for version control and collaboration. Created GitHub Project for easier delegation of tasks

2. Development Environment Configuration

Installed and configured VSCode, Xcode, and Android Studio

Set up Flutter and Dart development environment

Core System Architecture

1. Authentication System Implementation

- Login interface development with button and style design.
- Registration interface with return function.
- Account password matching functionality.

2. Role-Based User System

- Implemented student and vendor registration types.
- Created role-specific theme differentiation.
- Developed automatic redirection to appropriate homepages.

Features (Overall):

1. Registration Page

- User Identity Selection: Users can select different identities for registration (e.g., Student and Vendor).
- Back Button: A back button at the top left allows users to return to the login page if the wrong identity is selected.
- Double-Entry Password System: A double-entry password system ensures the password is correctly set before submission.

2. Login Page

- Username and Password Matching: Users must enter the username and password they set during registration. The system ensures only matching credentials grant access.
- Persistent Login: After logging out, users can log in again using the same credentials they set during registration.

3. Log out

- Users can securely sign out with a single tap on the red button, which clears all session data, redirects to the login page, and prevents back navigation for complete session termination.

3. Student Features

- Profile Picture Customisation: Users can select their profile picture from their photo gallery.

- Canteen Menu: Displays dish names and images. Users can click on a dish's image to enlarge it.
- Canteen & Dishes Review: Allows users to review canteens and dishes. Reviews can be modified or deleted. Both users and vendors can view reviews, and vendors can respond to them.
- Location Functionality: Users can share their location to help recommend canteens based on distance. Users can opt-in or out of location sharing.
- Canteen Sorting and Filtering: Users can sort and filter canteens based on preferences.
- Vendor Menu Update Notifications: Red dot notifications alert users when vendors update their menus.
- Announcement Bar: Displays important announcements.
- Favourites Feature: Users can mark their favourite canteens and filter based on those.
- Canteen Queue Estimation: Estimates queue times for canteens.
- Select Preferences: The student profile page includes a preference system that allows students to select multiple tags to indicate their majors, interests, and dietary preferences
- Fun interaction: These three features form an organic ecosystem rather than existing in isolation: dies & Chat Room & Games & Leaderboard
- Explore Users with Similar Preferences: Displays users with matching dietary preferences in a scrollable card list showing avatars, nicknames, and shared tags, accessible via the center "Explore" tab with magnifying glass icon.

4. Vendor Features

- Vendor Restaurant Management: Vendors can manage their restaurant profile and menu.
- Reply to Student Reviews: Vendors can respond to reviews for their restaurants.
- Add Dishes: Vendors can add new dishes to the menu.
- Vendor Dashboard: Displays three key metrics (today's reviews, total sales, pending orders) in color-coded cards with real-time updates, prioritizing visibility of urgent actions through strategic color use (green/orange/red) and responsive layout design.
- Quick Access Panel: Provides vendors with one-tap navigation to core functions (menu management, reviews, and analytics) through dashboard shortcuts, optimizing workflow efficiency with direct access to editing tools and performance metrics.
- Vendor Reviews System: A comprehensive dashboard enabling vendors to efficiently manage customer feedback through filtered views (highlighting unreplied messages), in-line reply functionality, and

real-time performance analytics (average rating/total count), all presented in an accessible color-coded interface.

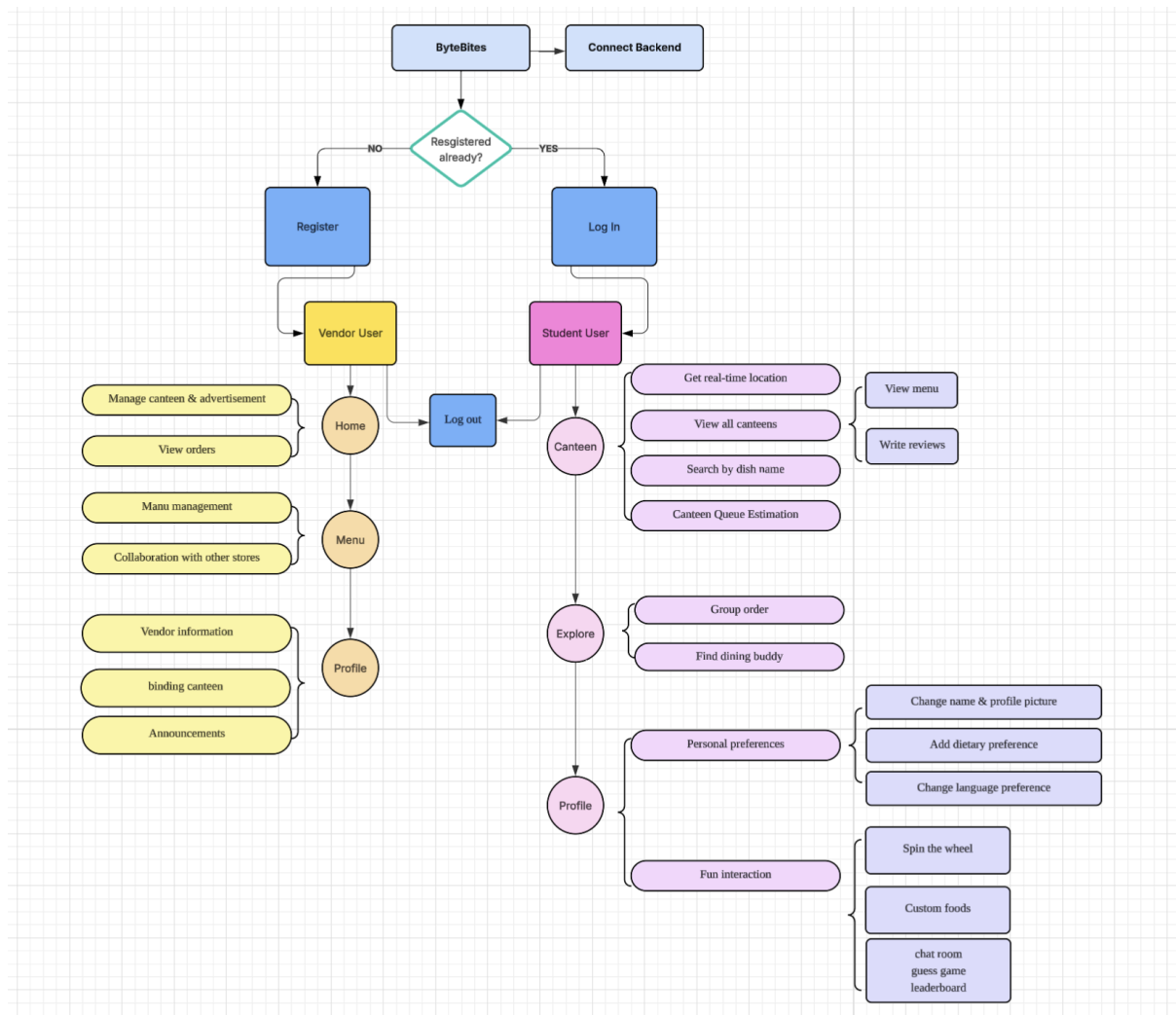
- Vendor Announcements: Enables canteen vendors to publish real-time banner notifications (special offers/schedule changes) that appear on student interfaces, with synchronized two-way data storage (vendor-specific and global lists) using JSON format and local persistence.

5. Additional Feature

- User Exploration: Explore Users with Similar Preferences: Switches to the Explore page, matching users with similar dietary preferences and displaying their info.
- Function Interaction: Random Spin Feature: Helps indecisive users quickly choose food or options.
- Saving User Language Preferences: Language Preference Saving: Saves the user's language choice using SharedPreferences (e.g., "zh" for Chinese, "en" for English).

Features Implementation

Software structure diagram



Registration & Login Features

1. Registration

Overview:

On the registration page, users can select different identities for registration, such as Student and Vendor. If the button is accidentally pressed, a back button at the top left allows users to quickly return to the login page. To prevent users from accidentally entering their password incorrectly, we have implemented a double-entry password system, ensuring that the password they set is accurate.

UI/UX Design:

1. Overall Layout:
 - Uses a black background (backgroundColor: Colors.black)
 - Features an AppBar with a back button at the top
 - The content area uses SingleChildScrollView to ensure all content is scrollable on smaller screens
 - All text and input fields use white font color, creating contrast against the black background
2. Main Components:
 - Large "Register" title (34pt font, bold)
 - Three text input fields: username, password, and confirm password
 - User type selection (student/vendor, using ChoiceChip)
 - Register button (green background, full width)
3. Input Field Design:
 - All input fields have white placeholder text
 - Password fields use obscureText: true to hide input
 - Input text is also white
4. User Type Selection:
 - Uses ChoiceChip components for visual feedback
 - The currently selected type is highlighted

Core Methods

1. isUsernameAvailable Method:
 - Checks if a username is available
 - Uses SharedPreferences to store user data
 - User data format: username|password|userType
 - Returns Future<bool> indicating whether the username is available
2. saveCredentials Method:
 - Saves new user credentials to SharedPreferences
 - Uses | as a delimiter to store username, password, and user type
 - Adds the new user to the existing user list
3. Register Button Logic:
 - Validates that password and confirm password match
 - Checks if the username is available
 - Saves credentials
 - Displays a success message and navigates to the login page

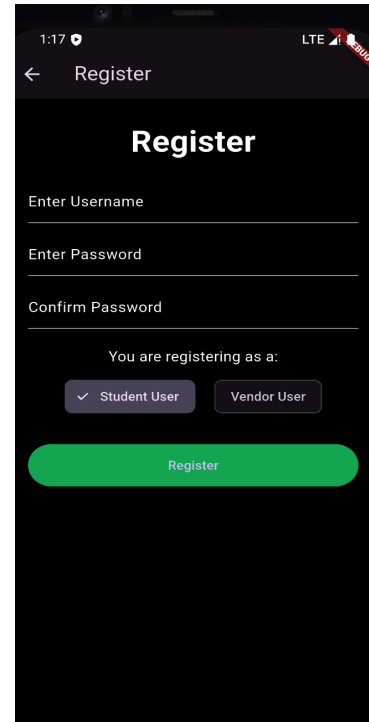
Interaction Design

1. Error Handling:
 - Shows a SnackBar when passwords don't match

- Shows a SnackBar when the username already exists
- 2. Success Feedback:
 - Displays a success message after registration
 - Automatically navigates to the login page
- 3. Navigation:
 - The back button in the AppBar returns to the previous page
 - After successful registration, uses `pushReplacementNamed` to replace the current route with the login page

Code Structure

1. StatefulWidget:
 - Uses StatefulWidget to manage form state and user interactions
2. Controllers:
 - Uses TextEditingController to manage text input
3. State Management:
 - Uses `setState` to update the user type selection state
4. Data Persistence:
 - Uses SharedPreferences as a simple local storage solution



2. Login

Overview:

On the login page, users need to enter the username and password they set during registration. We have implemented a username and password matching feature, allowing access only when the entered credentials match the ones set during registration. Additionally, after logging out, users can log in again using the same username and password they set, ensuring a seamless user experience.

UI/UX Design

1. Overall Layout:
 - Black background (`backgroundColor: Colors.black`)
 - Centered content with padding
 - `SingleChildScrollView` for scrollable content
 - Form widget for input validation
2. Main Components:
 - "ByteBites" app title (34pt bold white text)

- Optional user avatar circle (shown if `_userAvatar` exists)
 - Username and password input fields
 - Login button with SpongeBob image
 - Register button with Mr. Krabs image
3. Input Field Design:
- TextFormField for both username and password
 - Grey background (`Colors.grey[900]`) with white text
 - Outline borders
 - Password field obscures text
 - Red error text for validation messages

Core Methods

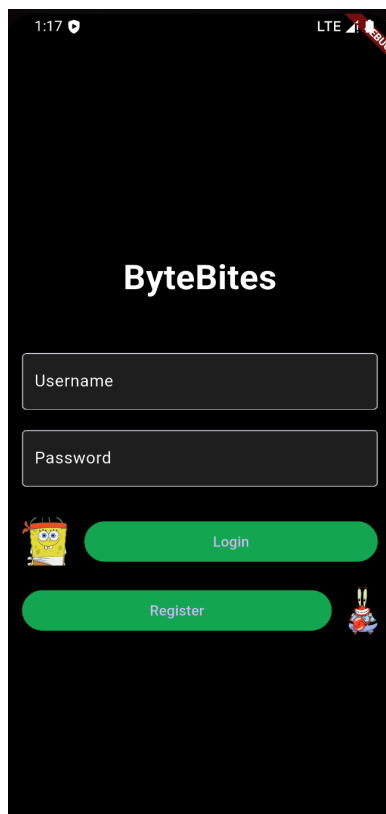
1. `_login()` Method:
- Validates form inputs first
 - Retrieves user list from `SharedPreferences`
 - Checks credentials against stored users
 - Handles successful/failed login attempts
 - Navigates to appropriate home screen based on user type

Key Features

1. Form Validation:
 - Uses Form and TextFormField for built-in validation
 - Checks for empty username/password fields
 - Displays red error messages below invalid fields
2. User Authentication:
 - Compares input against stored credentials
 - Supports both student and vendor user types
 - Handles avatar path if available in user data
3. Session Management:
 - Stores current user info in `SharedPreferences` for:
 - Username
 - User type
 - Avatar path (if exists)
4. Navigation:
 - Login button: Authenticates and navigates to appropriate home screen
 - Register button: Navigates to `RegisterScreen`
 - Uses `pushReplacement` for login to prevent back navigation
5. Visual Elements:
 - Fun character images (SpongeBob and Mr. Krabs) next to buttons
 - Conditional avatar display
 - Consistent green button styling (`Color(0xFF16a951)`)

Code Structure

1. StatefulWidget:
 - Manages form state, controllers, and avatar file reference
2. Form Handling:
 - GlobalKey for form validation
 - Validator functions for both fields
3. Data Flow:
 - Retrieves user data from SharedPreferences
 - Updates current session info upon successful login
 - Passes user to appropriate home screen



3. Log out

Overview

The logout feature allows users to securely sign out of their accounts by:

1. Clearing current user session data from SharedPreferences
2. Removing sensitive user information from memory
3. Redirecting back to the login screen
4. Preventing back navigation to authenticated screens

Code Structure Analysis

Functional Breakdown

1. Data Clearing:
 - `prefs.remove('currentUsername')` - Removes logged-in username
 - `prefs.remove('currentUserType')` - Removes user type (student/vendor)
 - `prefs.remove('currentUserAvatar')` - Removes profile image path
2. Navigation Control:
 - `Navigator.pushNamedAndRemoveUntil` with predicate (route) $\Rightarrow$ false ensures:
 - Redirects to login screen
 - Clears entire navigation stack
 - Prevents back button from returning to authenticated screens
3. Safety Check:
 - `if (mounted)` prevents state updates after widget disposal
4. Localization Support:
 - Uses `AppLocalizations` for translated logout text

UI/UX Design

1. Visual Design

- Button Styling:
 - Red color scheme (universal logout indicator)
 - Icon + text combination (logout icon for quick recognition)
 - Full-width button with minimum height (50dp for easy tapping)
- Positioning:
 - Placed at the bottom of profile pages
 - Separated from other content with top margin (30dp)

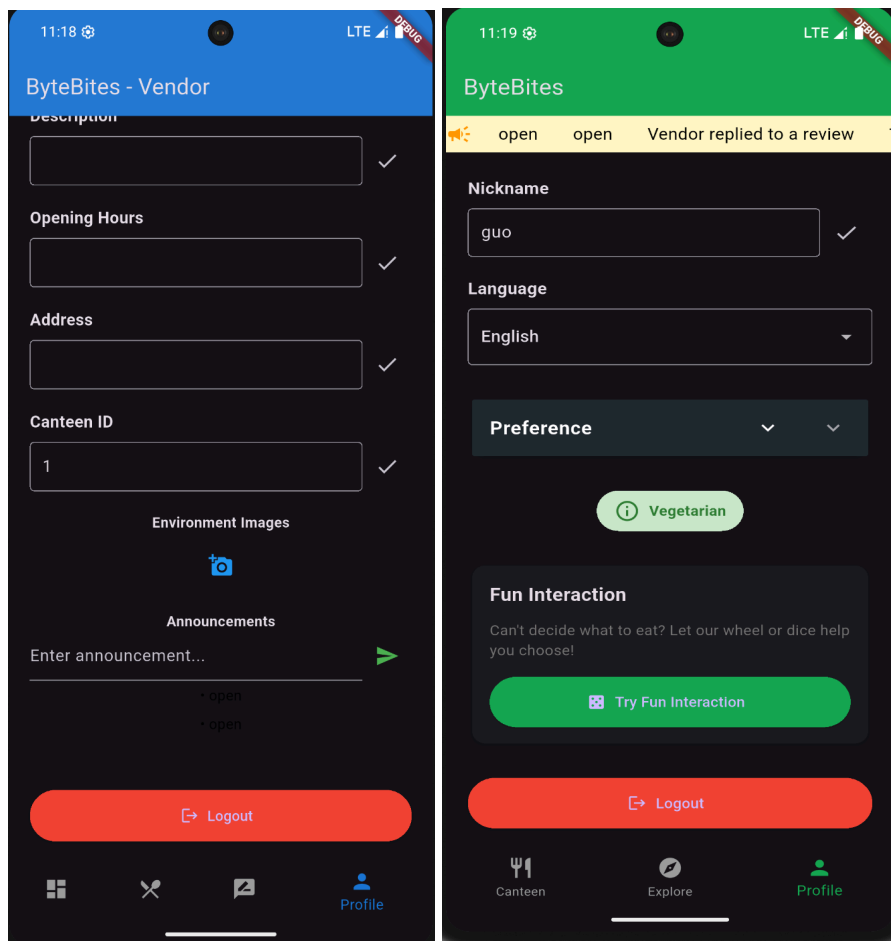
2. User Experience

- Clear Identification:
 - Standard logout icon
 - Red color conveys "destructive" action
 - Consistent placement across both user types
- Action Feedback:
 - Immediate navigation to login screen
 - No confirmation dialog (simple flow)
 - All session data properly cleared before redirect
- Security Considerations:
 - Comprehensive data removal
 - Navigation stack cleared

- Prevents back navigation to secured screens

Technical Highlights

- Complete Session Clearance:
 - Removes all user-specific data
 - Clears both required and optional user fields
- Safe Navigation Handling:
 - Uses `pushNamedAndRemoveUntil` with false predicate
 - Checks mounted state before navigation
- Consistent Implementation:
 - Same logic for both student and vendor users
 - `SharedPreferences` used as single source of truth
- Visual Consistency:
 - Same button styling across both user types
 - Standard Material Design logout icon



Student User Features

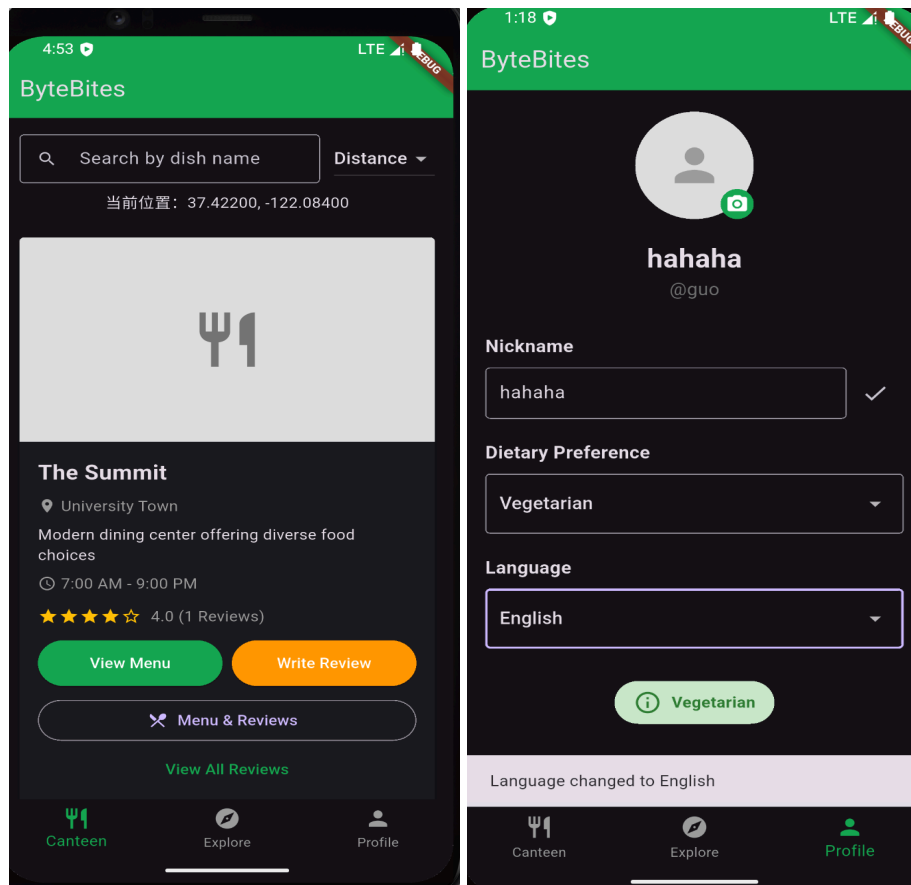
1. Bottom Navigation Bar Implementation:

Structural Composition

1. Page Management:
 - Uses IndexedStack to manage three pages:
 - CanteenPage: Dining hall page
 - ExplorePage: Explore page
 - ProfilePage: User profile page
 - Controls current page display via `_selectedIndex`
2. Navigation Bar Configuration:
 - Three navigation items:
 - Canteen (icon: `Icons.restaurant`)
 - Explore (icon: `Icons.explore`)
 - Profile (icon: `Icons.person`)
 - Selected state in blue, unselected in gray
3. Interaction Logic:
 - Tapping nav items triggers `_onItemTapped` method
 - Updates `_selectedIndex` and rebuilds UI via `setState`

Design Features

1. State Preservation:
 - IndexedStack maintains all page states without reconstruction during switches
2. Visual Consistency:
 - Matches navigation bar with AppBar using same blue color scheme
 - Icons and labels clearly communicate functionality
3. Extensibility:
 - To add new navigation items simply:
 1. Create new page component
 2. Add to `_pages` array
 3. Add new item to BottomNavigationBar's items



2. Profile picture change:

Overview:

Users can select their profile picture by choosing an image from their photo gallery. This feature is integrated with the photo library.

Implementation Principle

1. Avatar Selection:

- Uses the image_picker package to select images from device gallery
- ImageSource.gallery indicates selection from photo gallery
- Selected image is converted to a File object stored in _avatarImage

2. Avatar Storage:

- Saves avatar file path to SharedPreferences under currentUserAvatar key
- Simultaneously updates user list data by adding/updating avatar path (format: username|password|userType|avatarPath)

3. Avatar Display:

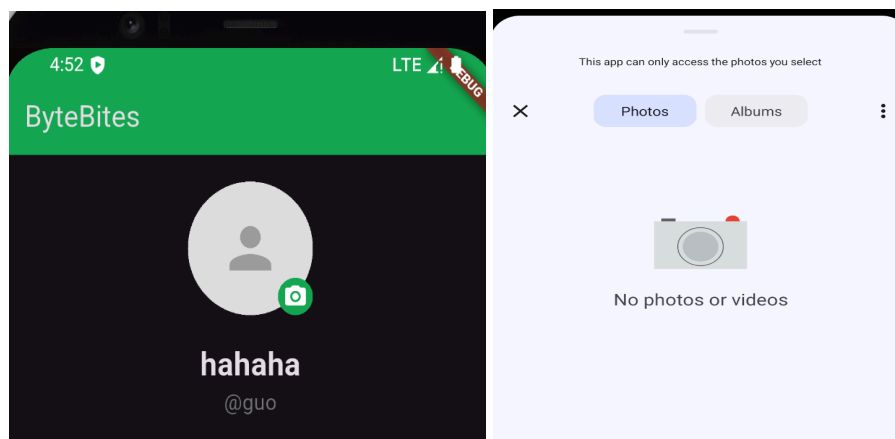
- Uses CircleAvatar component to display circular avatar
- If avatar file exists (_avatarImage not null), displays the file image
- If no avatar exists, displays default person icon
- Camera icon overlay in bottom-right corner indicates avatar can be changed

4. Interaction Design:

- Entire avatar area wrapped in GestureDetector that triggers avatar selection on tap
- Uses Stack to overlay avatar and camera icon
- Calls setState to update UI after selecting new avatar

Usage Flow

1. User clicks avatar area in profile page
2. System opens photo gallery selection interface
3. User selects an image
4. Image path is saved to local storage
5. Interface immediately updates to show new avatar
6. App loads and displays avatar path from storage upon next launch



3. Canteen Menu:

Overview:

The canteen menu displays the name and images of the dishes. Users can click on a dish's image to enlarge it.

Core Feature Architecture:

It implements a complete canteen menu browsing and dish review system, including:

- Menu display
- Dish review management (CRUD operations)
- Image viewing functionality
- User interaction interface

Data Management Implementation

Storage Solution:

- Uses SharedPreferences for local storage of all review data
- Data structure:
 - Each review stored as JSON string
 - Uses 'dish_reviews' key to store review list
 - Contains fields: dish name, rating, comment, user info, etc.

Image Viewing Functionality

Technical Solution:

- Uses InteractiveViewer for image zooming
- Clicking dish image pops up full-screen dialog
- Currently only "chicken rice" dish has sample image set

Review Management Features

CRUD Implementation:

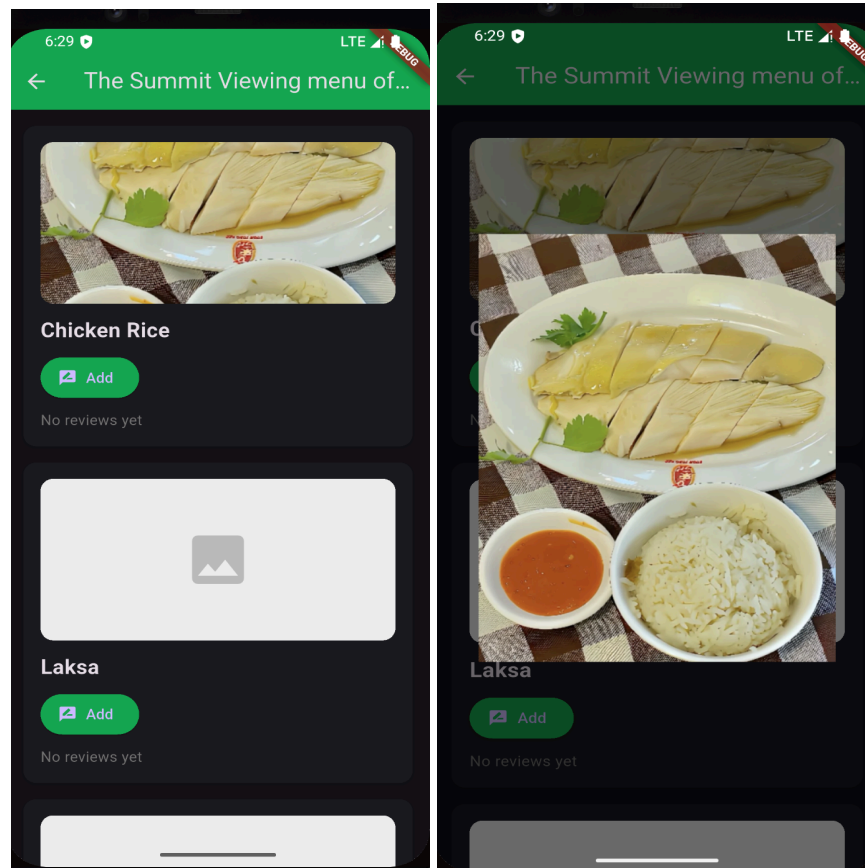
1. Add/Edit Reviews:
 - Dialog with star rating and comment input
 - Uses StatefulWidget for in-dialog state management
 - Form validation ensures non-empty comments
2. Delete Reviews:
 - Removes corresponding review from SharedPreferences
 - Immediately refreshes UI display

UI/UX Design Features

Visual Presentation:

1. Image Display:
 - Uniform 160px height containers
 - Rounded corners + clipping for visual consistency
 - Placeholder icon when no image available
2. Layout Alignment:
 - Uses Column and Padding for consistent spacing

- Review cards edge-aligned
 - Unified button styling
3. Interaction Design:
- Different colors distinguish add/edit actions
 - Intuitive star rating display
 - Operation buttons shown for user's own reviews



4. Canteen & dishes review:

Overview:

Allows users to review both canteens and individual dishes by rating them with stars. Reviews can be modified or deleted. Both users and vendors can see reviews, and vendors can respond to reviews for their restaurant.

Core Components:

1. Review Model (lib/models/review.dart):
 - Contains review ID, restaurant ID, username, nickname, rating, comment, and creation time

- Supports JSON serialization/deserialization
 - Facilitates local storage and retrieval
- 2. Review** Dialog
 (lib/screens/canteen/student/student_canteen_review_dialog.dart)
 - Star-based rating system (1-5 stars)
 - Comment input field (mandatory)
 - Form validation
 - Local storage for review data
- 3. Review** List Page
 (lib/screens/canteen/student/student_canteen_reviews_screen.dart)
 - Displays average restaurant rating
 - Chronologically sorted review list
 - User avatars and nicknames
 - Relative time display (e.g., "a few minutes ago," "hours ago")
- 4. Enhanced** Restaurant Card
 (lib/screens/canteen/student/student_canteen_card.dart)
 - Real-time average rating and review count
 - "Write a Review" button
 - "View All Reviews" button
 - Star-rating visualization

Key Features

1. Star-Rating System
 - 5-star rating scale
 - Visual star display
 - Supports half-star increments (for averages)
2. User-Friendly Interface
 - Intuitive rating selection
 - Clear review presentation
 - Responsive design
3. Local Data Storage
 - Uses SharedPreferences for review storage
 - JSON-formatted data
 - Data persistence
4. Multi-Language Support
 - English and Chinese interfaces
 - Dynamic language switching
 - Complete localized strings
5. Real-Time Updates
 - Instant display updates after submission
 - Live calculation of ratings and counts
 - Data refresh upon page return

User flow:

View Restaurants

- Open the Restaurant List page.
- Each restaurant card displays:
 - Basic info (name, location, etc.)
 - Average rating (star display)
 - Number of reviews
 - "Write a Review" and "View All Reviews" buttons.

Write a Review

- Tap "Write a Review" on a restaurant card.
- A rating dialog appears with:
 - Star selection (1-5 stars) – Tap to rate.
 - Comment field – Write your thoughts (required).
 - Validation – Ensures a rating and comment are provided.

Submit the Review

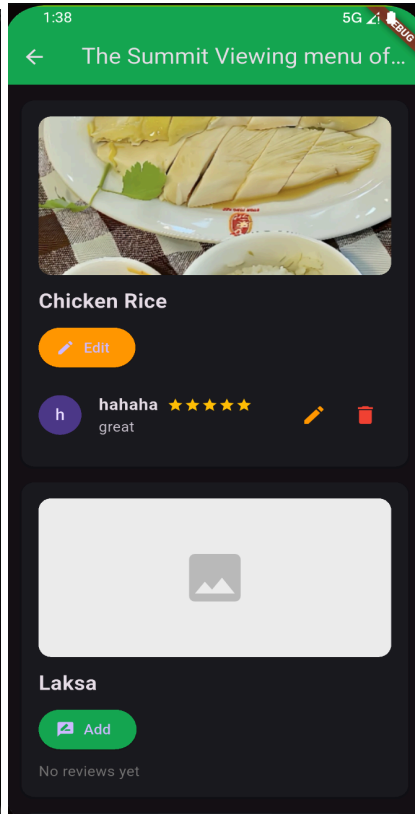
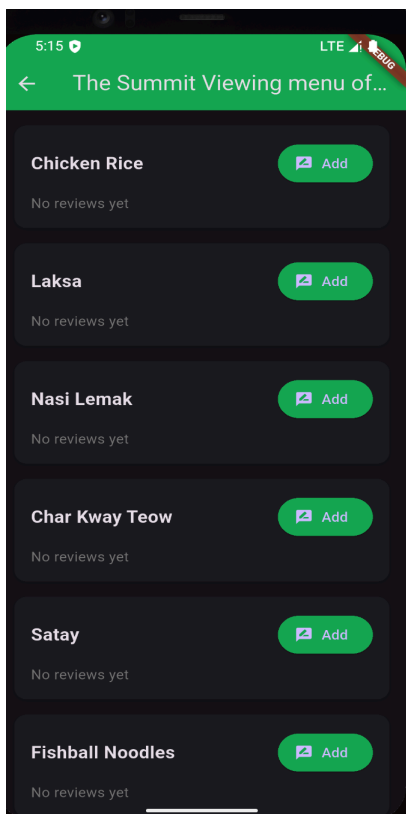
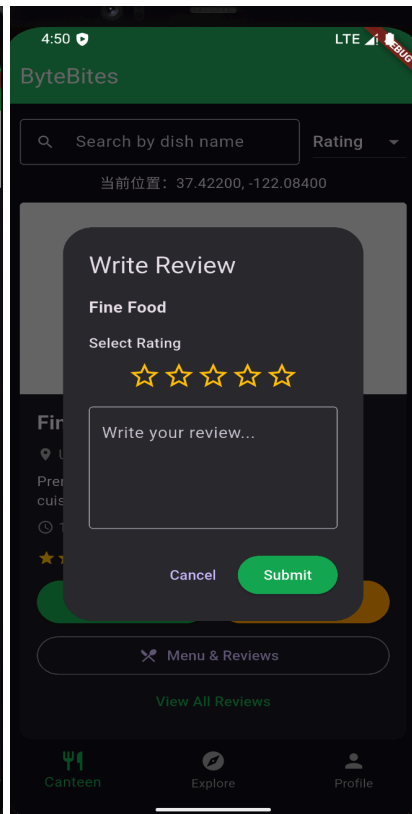
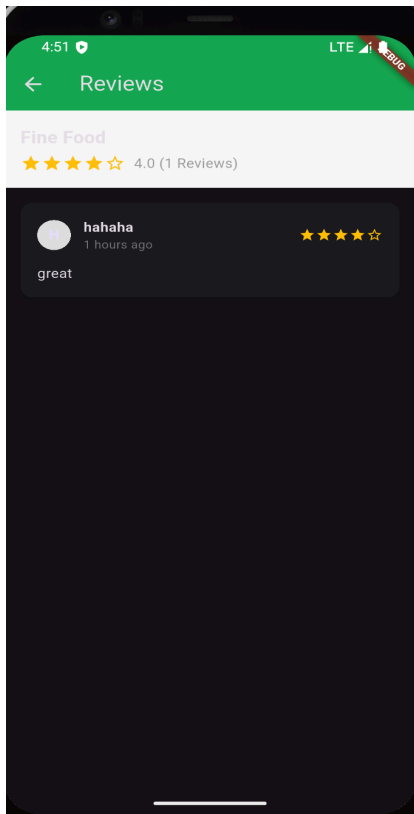
- Click "Submit" to save the review.
- The review is stored locally (via SharedPreferences).
- A success message confirms submission.

View All Reviews

- Tap "View All Reviews" on the restaurant card.
- A dedicated reviews page shows:
 - Average rating at the top.
 - List of all reviews, sorted by time (newest first).
 - Each review includes:
 - User's nickname & avatar
 - Star rating
 - Comment
 - Time posted (e.g., "2 hours ago").

Real-Time Updates

- After submitting a review:
 - The average rating recalculates instantly.
 - The review count updates.
 - The restaurant card refreshes to reflect changes.
 - The reviews list automatically includes the new entry.



5. Location Functionality:

Overview:

Users can allow the app to access their location, helping to recommend canteens based on distance. Users can opt-in or out of sharing their location, with language-specific prompts when making these choices. Additionally, location data helps estimate restaurant traffic.

Implementation Overview

1. Dependency Integration:
 - Uses the geolocator plugin for location services
 - Added to pubspec.yaml with: geolocator: ^latest_version
2. Permission Handling Workflow:
 - Checks if location services are enabled on the device
 - Requests user permission if not already granted
 - Handles various permission states (granted/denied/restricted)
3. Location Acquisition:
 - After permission granted, retrieves current coordinates
 - Stores location data in _userPosition variable
 - Uses high accuracy mode for best results
4. Application Usage:
 - Calculates restaurant distances using the coordinates
 - Enables location-based sorting functionality
 - Displays location-aware content to users

Error Handling

1. User Denial Cases:
 - Gracefully handles when users deny permissions
 - Provides friendly UI messages explaining the need for location
 - Offers guidance to re-enable in settings
2. Service Disabled:
 - Detects when device location services are off
 - Guides users to enable through system settings

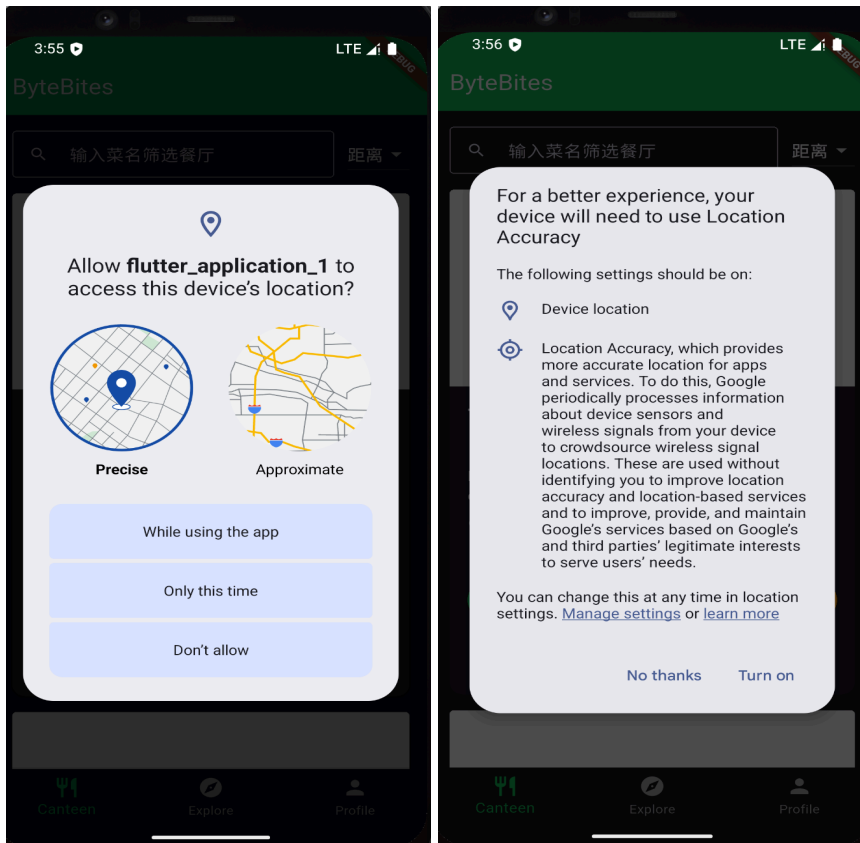
OS Configuration:

- Android: Requires these permissions in AndroidManifest.xml:

```
<uses-permission
android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission
android:name="android.permission.ACCESS_COARSE_LOCATION" />
```
- iOS: Needs these entries in Info.plist:

```
<key>NSLocationWhenInUseUsageDescription</key>
<string>Need location to show nearby restaurants</string>
```

```
<key>NSLocationAlwaysUsageDescription</key>
<string>Need location to show nearby restaurants</string>
```



6. Canteen Sorting and Filtering:

Sorting Function Implementation:

Technical Solution:

- State Management: Uses Flutter's `StatefulWidget` and `setState()` to manage sorting states
- Data Source Processing: Maintains two lists
 - `_allCanteens`: Stores original restaurant data
 - `_filteredCanteens`: Stores filtered and sorted results
- Sorting Trigger Mechanism: Activated via dropdown menu's `onChanged` callback

Specific Sorting Methods:

- Distance Sorting:

- Obtains device GPS coordinates using geolocator plugin
- Calculates spherical distance using Haversine formula
- Dynamically computes distances between user location and restaurants during sorting
- Rating Sorting:
 - Retrieves historical rating data from SharedPreferences
 - Parses rating records using jsonDecode
 - Dynamically calculates average ratings for each restaurant
- Name Sorting:
 - Uses Dart's built-in string comparison methods
 - Supports multilingual name sorting

Dish Search Function Implementation

Core Technology:

- String Matching: Uses String.contains() for dish name matching
- Case Handling: Converts to lowercase for comparison (toLowerCase())
- Real-time Search: Triggered instantly via TextField's onChanged callback

Filtering Process:

- Gets search box input
- Iterates through all restaurants' menu lists
- Checks if any dish contains the search keyword
- Builds new filtered results list

Combined Functionality Implementation

Execution Order:

- Applies dish filtering first (if search term exists)
- Then applies sorting to filtered results
- Finally updates UI display

Key Technical Points:

- Uses unified _applyFilterAndSort() method for processing
- Maintains two state variables: _searchText and _sortType
- Triggers UI updates via setState()

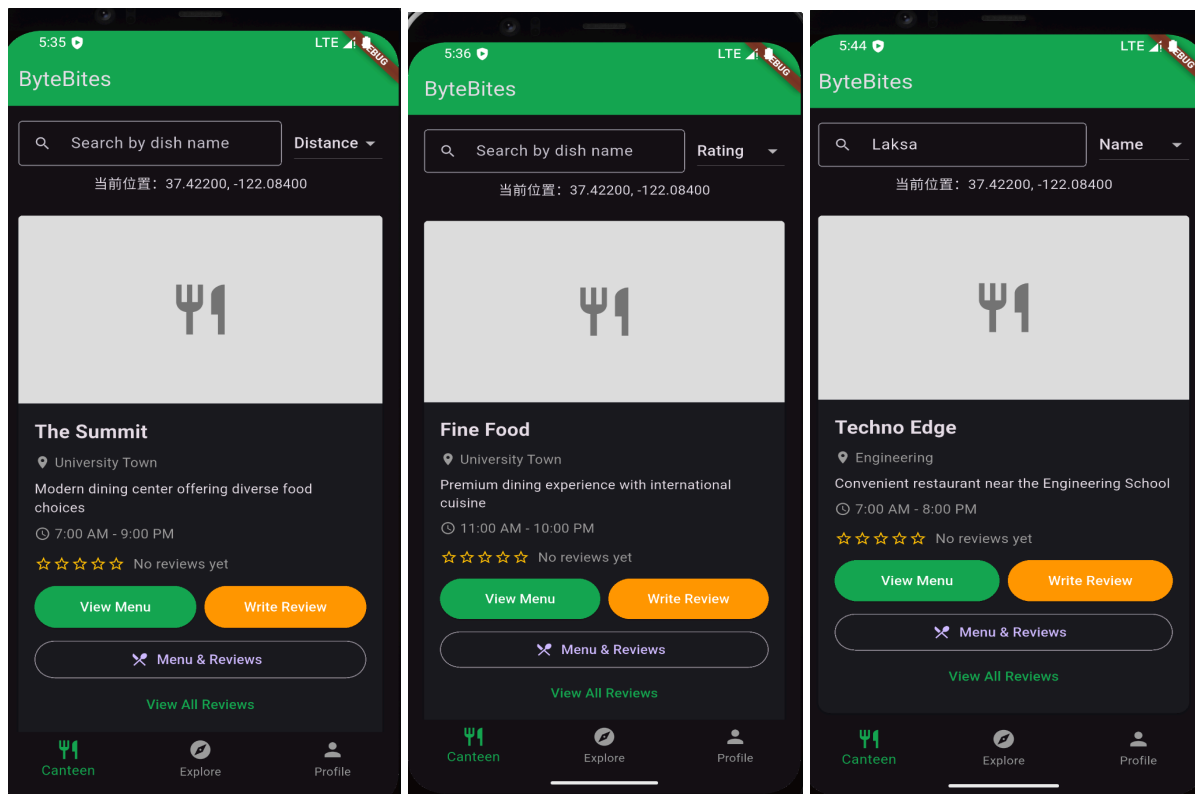
Key Technical Dependencies

- Location Services: geolocator plugin
- Local Storage: SharedPreferences
- State Management: Flutter's native State management
- UI Components: Material Design component library

Distance:

Rating:

Name:



7. Vendor Menu Updates Trigger Red Dot Notifications:

Overview:

Vendors can update their menus, and students will be notified via red dot notifications on the app.

When a vendor updates a canteen's menu (e.g., adding dishes or adjusting prices), the system automatically marks the canteen as "unread" in local storage

When the student opens the app and views the canteen list:

- The system checks each canteen's unread status.
- If updates exist, a red dot appears in the top-left corner of the canteen card

When the student taps the canteen card, The red dot vanishes, confirming the user has seen the updates

Core expression:

1. State Storage Mechanism

- Uses SharedPreferences for local storage to track the "unread" status of each canteen.
- Stores a boolean flag for each canteen with the key format: "canteen_unread_[CanteenID]".
 - Example: The unread status of Canteen 1 is stored under the key "canteen_unread_1".

2. State Detection & Display

- When the canteen card initializes (initState), it calls _checkUnread().
- This method reads the unread status from SharedPreferences.
- The _hasUnread variable controls whether the red dot is shown/hidden.
- The red dot is positioned absolutely (Positioned) in the top-left corner of the card.

3. State Update Logic

- When the vendor modifies the menu, the corresponding canteen's unread flag is set to true.
- The student app checks these flags every time it launches.
- If a flag is true, the red dot is displayed as a reminder.

4. State Clear Mechanism

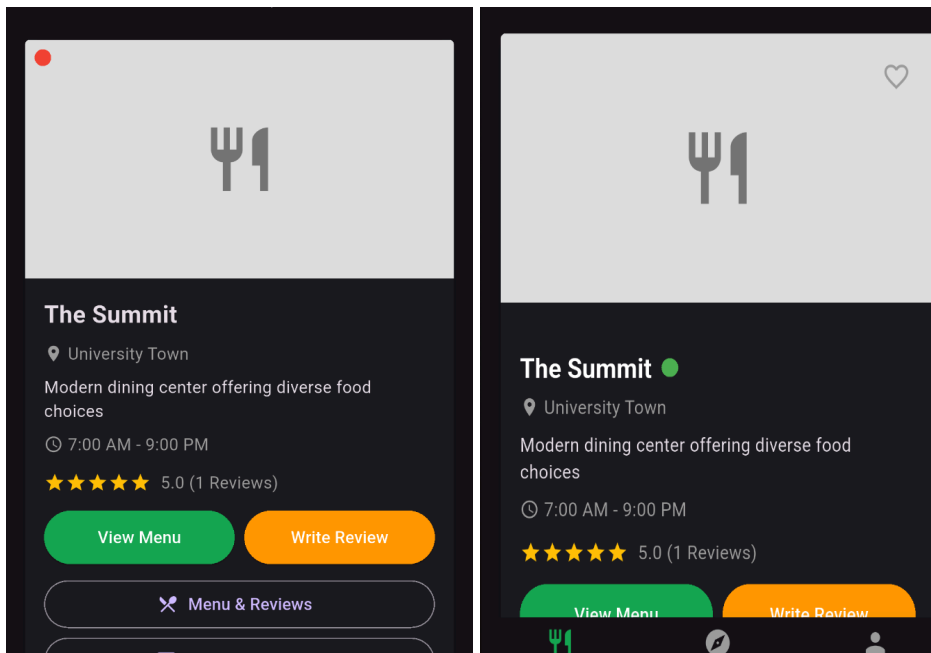
- When the user clicks to view the menu, _clearUnread() is called.
 - This resets the canteen's unread flag to false.
 - Updates _hasUnread to trigger a UI refresh.
 - The red dot disappears, indicating the user has seen the latest updates.

UI Design Details

- Red dot:
 - A 14-pixel diameter red circle (Container with BoxShape.circle and Colors.red).
 - Positioned at left:8, top:8 (does not obstruct content).
- Implemented using Stack + Positioned for overlay placement.

***update**

*** after clicking, red dot disappear**



8. Announcement Bar:

Overview:

New announcements are automatically pushed when canteens update menus or reply to reviews (e.g., `_addAnnouncement` in `vendor_menu_page.dart`). Unread markers disappear after the user views the corresponding canteen (referenced in `_clearUnread` in `student_canteen_card.dart`).

Implementation Method

Component Location:

- The announcement bar is embedded as the bottom property of the AppBar, with its height controlled via `PreferredSize`.
- It uses a custom `AnnouncementBar` component (imported from `widgets/announcement_bar.dart`).

Data Storage:

- Announcement data is stored in `SharedPreferences` under the key 'announcements' (referenced in `_addAnnouncement` methods in `vendor_menu_page.dart` and `vendor_reviews_page.dart`).
- Each announcement includes the message content, canteen ID, timestamp, etc., formatted as a JSON string.

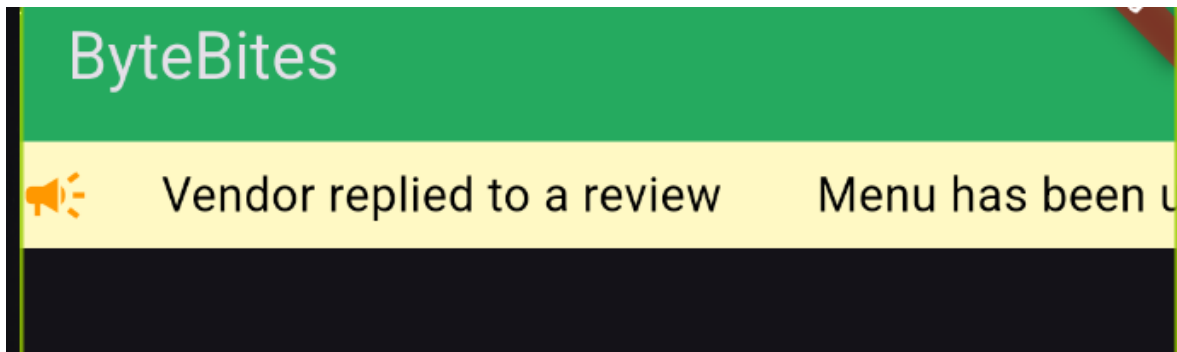
Unread Markers:

- Unread status is tracked using the key (referenced in the `_saveDishes` method in `vendor_menu_page.dart`).

UI/UX Design

Interaction Logic:

- Auto-scrolling: Announcement content likely loops in a marquee-style display.
- Unread Notifications: A red dot appears in the top-right corner of canteen cards (referenced in the `_hasUnread` logic in `student_canteen_card.dart`).



9. Favorites Feature:

Overview:

The favorites feature is a core interactive element in the student canteen list application, allowing users to bookmark preferred canteens for quick access.

Data Storage Architecture

Storage Mechanism:

- Utilizes Flutter's `SharedPreferences` plugin for persistent storage
- Stores data as key-value pairs with the key `favorite_canteen_ids` and a list of strings as the value
- Each canteen is uniquely identified by an ID to ensure precise targeting

Core Logic Implementation

State Management:

- Uses `_favoriteCanteenIds` set to store currently favorited canteen IDs
- Employs Flutter's built-in `setState` mechanism for state management
- The `Set` data structure ensures unique IDs and fast lookups ($O(1)$ complexity)

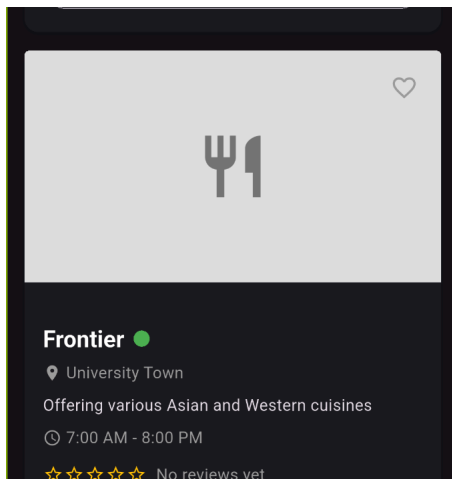
Initialization Process:

1. Calls `_loadFavorites` during component initialization
2. Asynchronously reads the favorites list from `SharedPreferences`
3. Converts results to a `Set` and updates component state

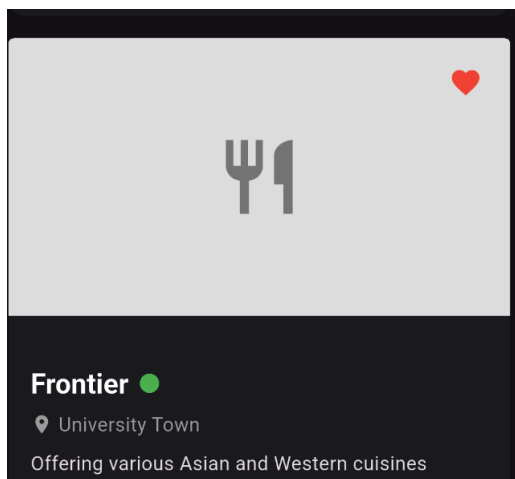
User Experience Flow

1. Discovery Phase:
 - User browses canteen list
 - Identifies favorite feature via heart icon
 - Hover tooltips display functionality explanations
2. Interaction Phase:
 - Clicks heart icon to toggle favorite status
 - Receives instant visual feedback (color change + icon switch)
 - Data automatically saves to local storage
3. Filtering Phase:
 - Selects "Favorite Only" from dropdown filter
 - List dynamically updates to show only favorited canteens
 - State persists across sessions

***before 'favorite'**



***after, heart become red**



10. Filter Favorited Canteens:

Filter Logic

1. Data Storage

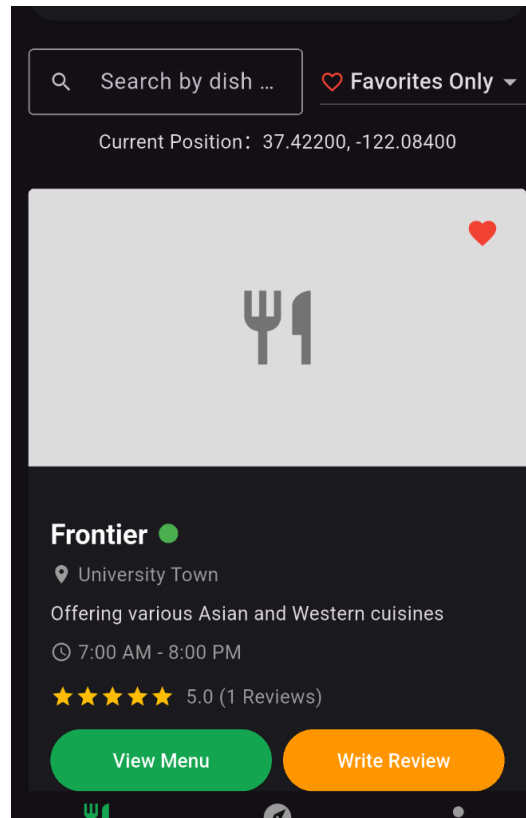
- When a user clicks the heart icon on a canteen card, the canteen ID is saved locally in SharedPreferences (under the key `favorite_canteen_ids`).
 - Favorited canteens display a solid red heart, while non-favorited ones show a hollow gray heart.
2. Triggering the Filter
- When the user selects "Favorites Only" from the dropdown menu:
 - The system checks the `_favoriteCanteenIds` set (which stores all favorited IDs)
 - Filters the full canteen list to only show entries with matching ids
 - The list updates in real-time to display only favorited canteens
3. Empty State Handling
- If no canteens are favorited, a prompt is displayed (e.g., "No favorited canteens yet").

Function Entry Point

- Location: Located in the top navigation bar of the canteen list page
- Control: A dropdown menu (DropDownButton) with the following options:
 - Sort by distance
 - Sort by rating
 - "Favorites Only" (displayed with a red heart icon)

User Interaction Flow

1. Marking Favorites
 - Browse the canteen list → Click the heart icon in the top-right corner of a card → The icon turns red (favorited)
2. Filtering Favorites
 - Click the dropdown menu at the top → Select "Favorites Only" → The list immediately updates to show only favorited canteens
3. Canceling the Filter
 - Open the dropdown menu again → Select another option (e.g., "Sort by distance") → The full list is restored, with favoured canteens still marked in red



11. Canteen Queue Estimation (an imperfect function now):

Overview:

The queue estimation functionality is primarily implemented in `canteen_queue_page.dart` and integrated into the main canteen list view in `student_canteen_list.dart`. The system provides real-time queue status indicators for campus canteens based on user proximity and time of day.

Principles

1. Distance-based prediction model:
 - The system calculates real-time linear distance between users and each canteen
 - Employs professional geolocation calculation formula (Haversine formula) to ensure accuracy
 - Closer distances indicate higher probability of users arriving at the canteen, corresponding to increased queue probability
2. Time factor adjustments:
 - The system automatically identifies peak dining hours (11:00-13:00)

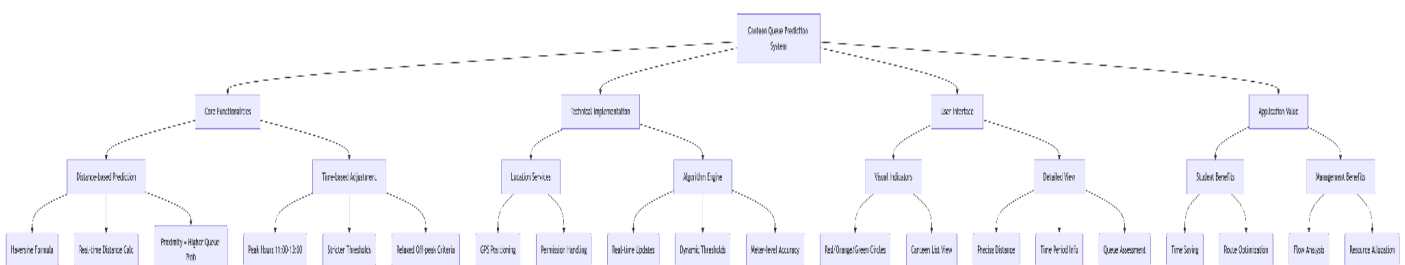
- Stricter judgment criteria are applied during peak hours
- Overall queue probability assessment is reduced during off-peak periods

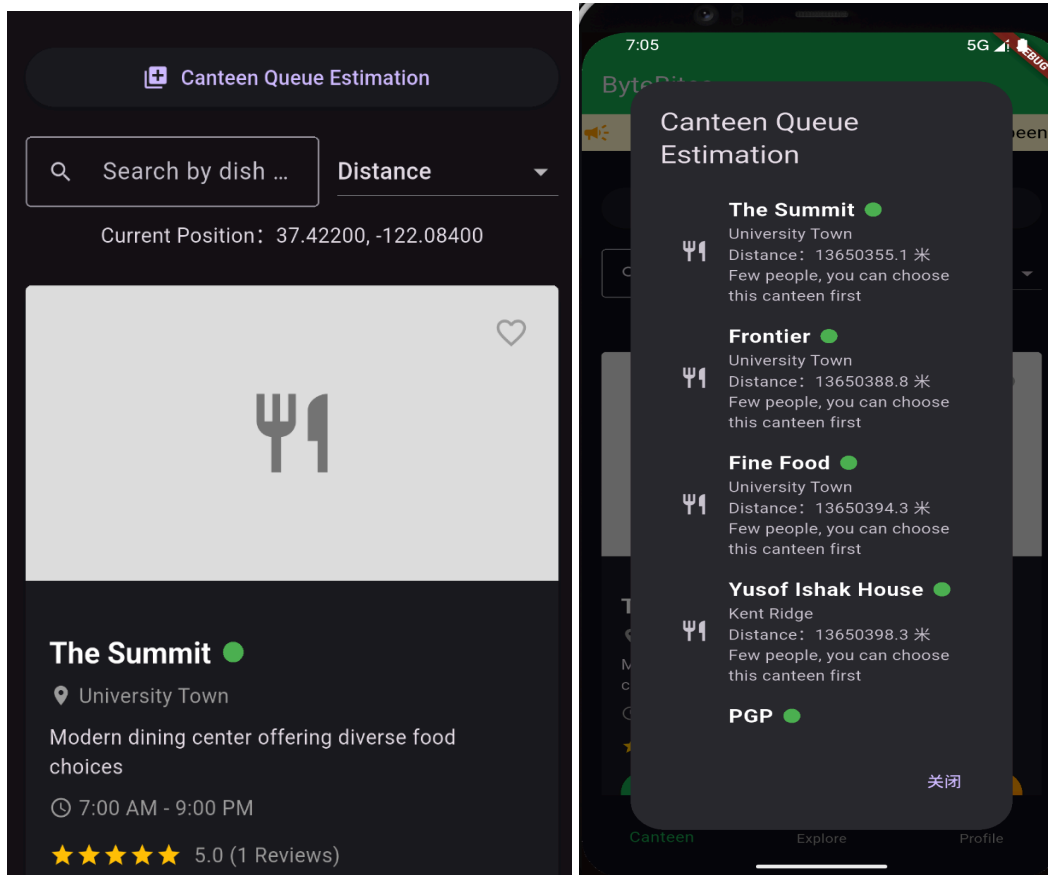
User Interface Design

1. Intuitive visual indicators:
 - Red circle: Indicates high queue risk (crowded)
 - Orange circle: Indicates moderate queue conditions
 - Green circle: Indicates low queue risk (available)
2. Information display methods:
 - Color-coded indicators appear beside each canteen name in the list view
 - Detailed prediction data available by clicking "Queue Estimation" button
 - Detailed information includes: precise distance, current time period, specific queue assessment
3. The system applies different distance thresholds depending on the time of day:
 - Peak Hours (11:00-13:00):
 - <100m: Red (High probability)
 - 100-300m: Orange (Medium probability)
 - 300m: Green (Low probability)
 - Non-Peak Hours:
 - <100m: Orange (Medium probability)
 - 100m: Green (Low probability)

Technical Implementation Details

1. Location services:
 - Uses mobile GPS to obtain real-time user location
 - Automatically requests location permissions
 - Provides guidance for enabling location services
2. Intelligent algorithms:
 - Distance calculation accurate to meter-level
 - Dynamically adjusted evaluation thresholds





12. Language Preferences:

Saving User Language Preferences

- Each user's language choice gets remembered (just like saving your login credentials)
- Uses the phone's built-in storage (SharedPreferences)
- Stores language codes like "zh" for Chinese and "en" for English

The Language Switching Function

- There's a dedicated variable (`_locale`) tracking the current language
- Just call the `changeLanguage` method to switch languages
- The whole app updates immediately when changing languages

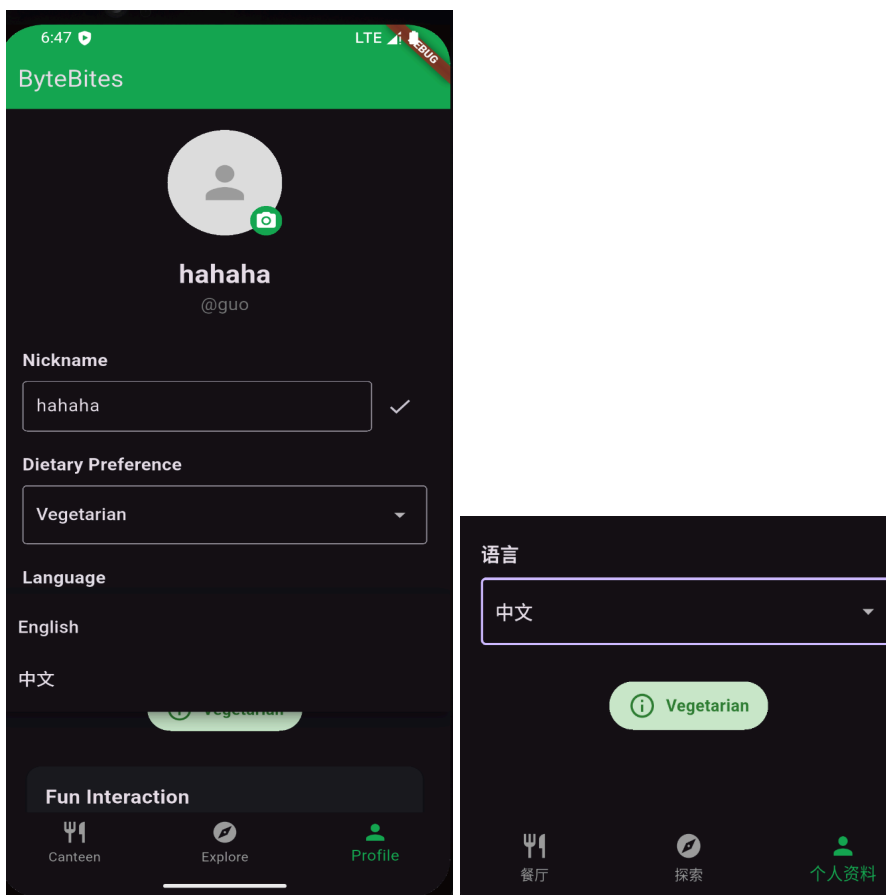
Managing Translated Text

- All translatable text is stored in special files (ARB files)
- Works like a dictionary with translations for each phrase

- Just "look up" this dictionary when you need translations

Language Switching Guide

1. Go to Settings
 - Open the App → Tap "Profile" at bottom right
 - Find "Language"
2. Select Language
 - Choose your preferred language (e.g. "简体中文", "English")
 - Interface changes immediately
3. Auto-Saved
 - App will remember your choice next time
 - Can change anytime



13. Select Preferences:

Overview:

The student profile page includes a preference system that allows students to select multiple tags to indicate their majors, interests, and dietary preferences. These preferences are used for:

1. Matching system - Finding other students with similar preferences in the "Explore" page
2. Personal display - Showcasing student interests and characteristics in their profile
3. Dining recommendations - Recommending suitable canteen options based on dietary preferences

1. State Management Section

The state management of the preference system is primarily concentrated in the `_ProfilePageState` class, consisting of the following key components:

- Tag Option Definitions: Defines three categories of tag option arrays
 - majorOptions: Major options
 - interestOptions: Interest options
 - dietaryOptions: Dietary preference options
- Selected Tag Storage: Uses three arrays to store user selections
 - selectedMajors: Selected majors
 - selectedInterests: Selected interests
 - selectedDietaries: Selected dietary preferences
- Multilingual Mapping: Provides multilingual tags through three getter methods
 - majorOptionsMap: Multilingual mapping for majors
 - interestOptionsMap: Multilingual mapping for interests
 - dietaryOptionsMap: Multilingual mapping for dietary preferences

2. Data Persistence Section

Data loading and saving are implemented using `SharedPreferences`:

- Data Loading:
 1. Reads the current username from `SharedPreferences`
 2. Loads selected tags for all three categories
 3. Updates component state using `setState`
- Data Saving:
 1. Obtains `SharedPreferences` instance
 2. Saves the three tag arrays to storage using the username as the key
 3. Uses `await` to ensure completion of saving

3. UI/UX Design

1. Visual Design

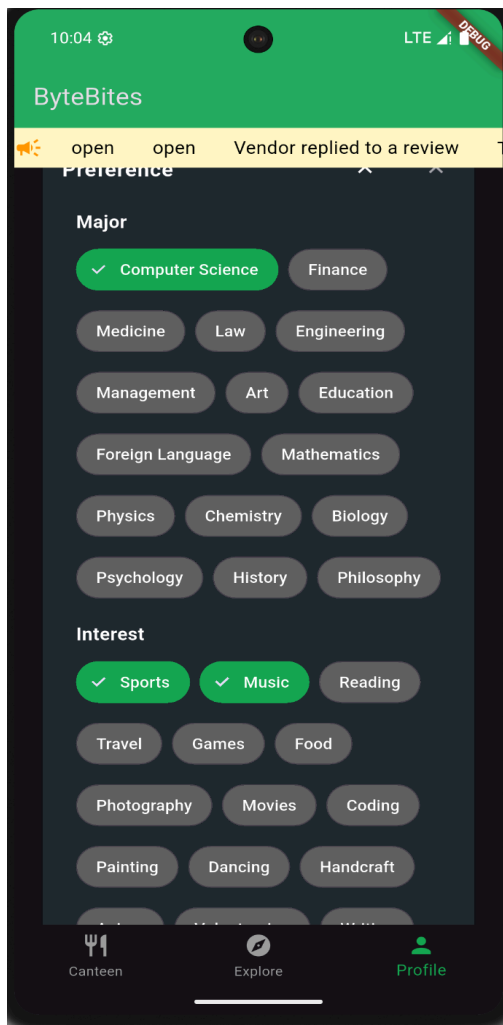
- Collapsible Panel: Utilizes Material Design's `ExpansionPanelList` to save space while maintaining full functionality.
- Tag Selection: Implements `ChoiceChip` components with color-changing feedback for selected states.
- Dietary Badge: Selected dietary preferences are prominently displayed at the top of the profile.

2. User Experience

- Intuitive Operation: Users can toggle tag selection states with a single click.
- Instant Feedback: UI updates and saves selections immediately after interaction.
- Multilingual Support: Displays tags in the user's preferred language.
- Categorized Management: Preferences are divided into three groups—majors, interests, and dietary—for easier comprehension and use.

3. Interaction Flow

1. The user clicks the "Preferences" header to expand the panel.
2. Available tags are browsed under each category.
3. Clicking a tag adds it to the profile.
4. Tags visually change to a selected state and are saved instantly.
5. Dietary preferences receive additional visibility at the top of the profile.



14. Explore Users with Similar Preferences:

Data Structure:

- Each matched user stored as Map containing:
 - username: Username
 - nickname: Display name (fetched from prefs, defaults to username)
 - avatarPath: Profile picture path
 - commonPreferences: List of shared preferences

UI Presentation

1. Loading States:
 - Shows CircularProgressIndicator during data loading
 - Displays special empty state UI (with icon and message) when no matches found
2. User List:
 - Uses ListView.builder for efficient scrollable list

- Each user item contains:
 - Left circular avatar (using CircleAvatar)
 - Top-center displays nickname (bold)
 - Bottom-center shows username (with @ prefix)
 - Bottom displays all shared preference tags (using Wrap and Chip for flow layout)
- 3. Interactions:
 - Tapping user item shows SnackBar (view details functionality pending)

Feature Placement

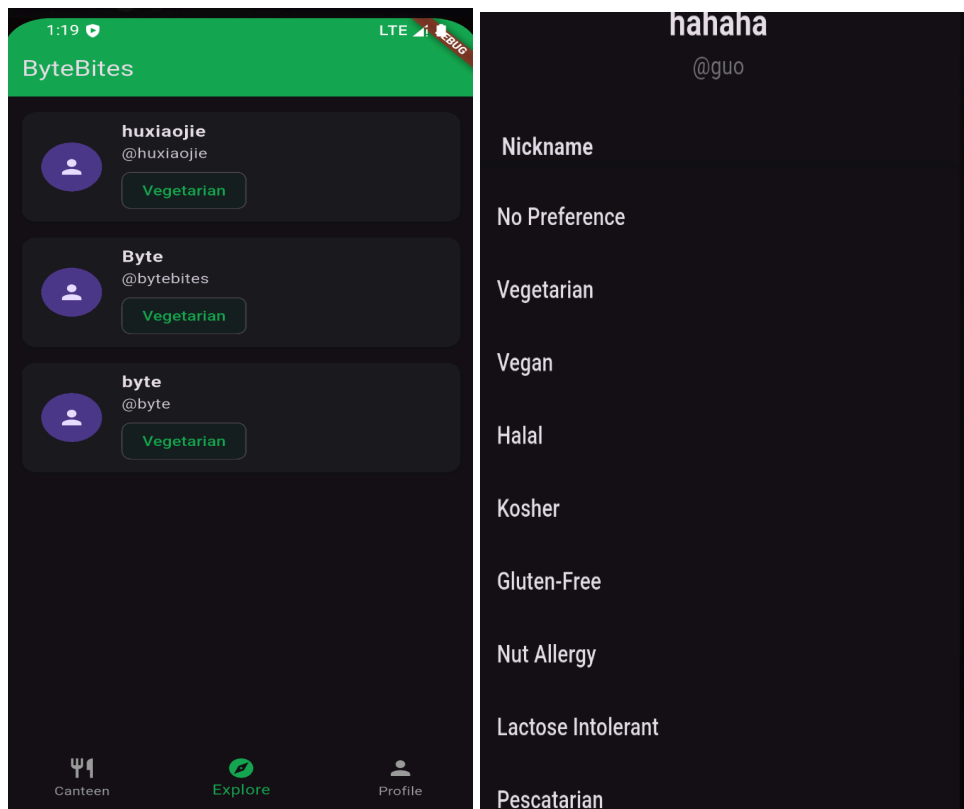
- Added an "Explore" option in the middle position of the bottom navigation bar
- Positioned between the "Dining" and "Profile" functions

Interface Design

- Uses a magnifying glass icon to represent the explore feature
- Clearly labeled as "Explore" for intuitive understanding
- Highlights in blue when selected, maintaining consistent styling with other navigation options

Functionality Implementation

- Switches to the Explore page when clicked
- Automatically searches for other users with similar dietary preferences upon loading
- Displays matched users in card format including avatars, nicknames and common preference tags



15. Function Interaction:

Technical Implementation Features

1. State Management:
 - Uses setState for UI state changes
 - Tracks current spinning/rolling state to prevent duplicate triggers
 - Records current wheel angle and dice number
2. Custom Painting:
 - Implements wheel drawing using CustomPainter
 - Dynamically calculates sector areas and text positions
 - Handles special cases for empty lists
3. Multilingual Support:
 - All interface text retrieved via AppLocalizations
 - Supports Chinese/English switching
4. User Isolation:
 - Uses username as key for custom food storage
 - Ensures data separation between different users

Main Interactive Components

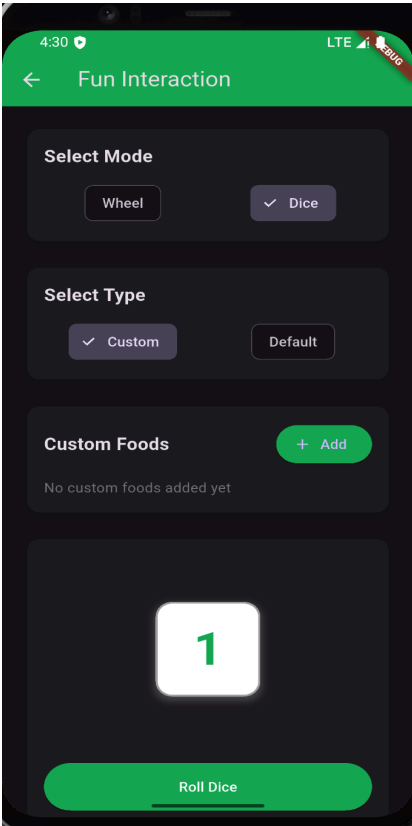
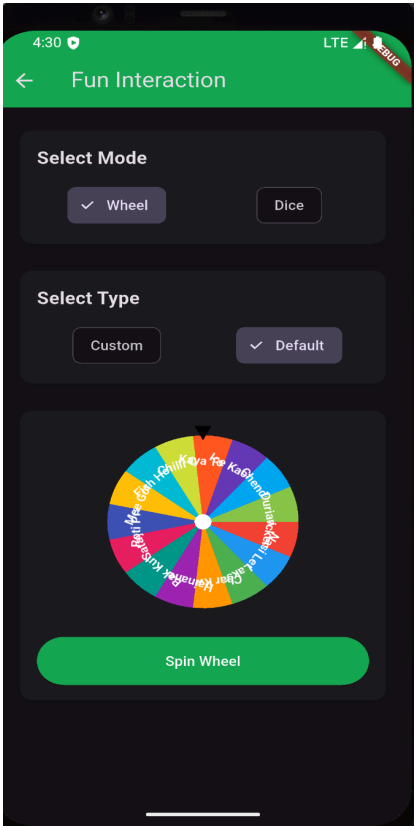
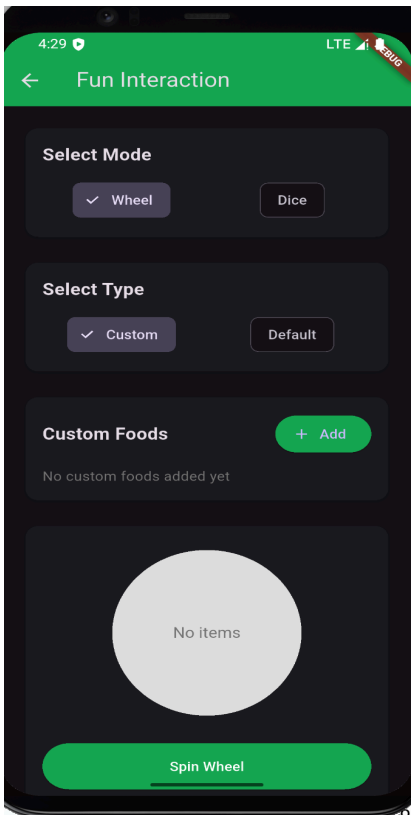
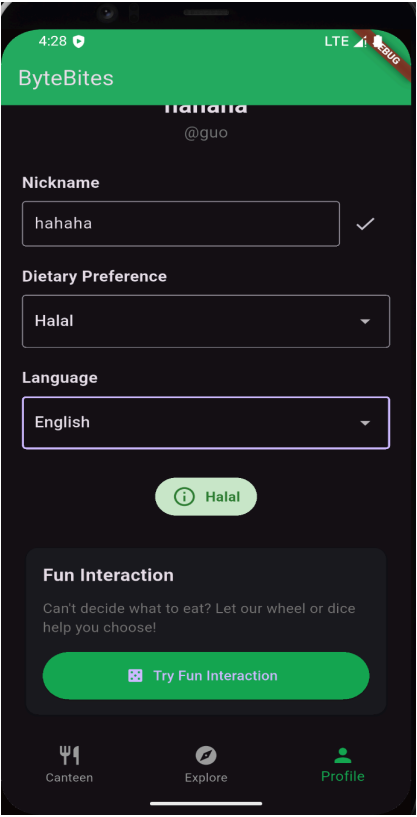
1. Wheel Component:
 - Colorful pie-chart wheel with each sector displaying a food name
 - Clicking the "Spin" button triggers a spinning animation with 5-10 random rotations before landing on a food item
 - Smooth easing animation effect during spinning
 - Pointer at the wheel center indicates final selection
2. Dice Component:
 - 3D-style dice display showing numbers 1-6
 - Clicking the "Roll Dice" button triggers a bouncing animation
 - Dice bounces 3-6 times before showing final number
 - Random restaurant selection based on dice result

Design

1. Mode Selection:
 - Top card provides "Wheel" and "Dice" mode options
 - Uses ChoiceChip component for single selection
 - Corresponding interactive component displayed below after selection
2. Type Selection:
 - Second card offers "Custom" and "Default" data source options
 - Shows food management interface when "Custom" is selected
3. Custom Food Management:
 - Displays all custom foods for current user
 - Each food shown as a deletable Chip component
 - "+Add" button opens dialog for new food entry
 - Empty state shows "No custom foods" message
4. Result Display:
 - Popup dialog shows randomized selection result after interaction
 - Displays selected food/restaurant name
 - Provides "OK" button to close dialog

Animation Effects

1. Wheel Animation:
 - Uses Transform.rotate for spinning effect
 - 3-second animation duration with easeOutCubic curve
 - Automatically shows result when spinning ends
2. Dice Animation:
 - Uses Transform.scale for bouncing effect
 - 500ms animation duration with easeOutBack curve
 - Shows final number and result when bouncing ends



15.1 Improvement of fun interaction

Overview:

These three features form an organic ecosystem rather than existing in isolation:

1. Chat Room & Games: Users can discuss number-guessing strategies and share gaming experiences in chat
2. Games & Leaderboard: Game performance directly affects leaderboard, creating positive reinforcement
3. Leaderboard & Social: Leaderboard provides chat topics, enhancing user interaction

1. Chat Room Functionality

Core Experience: The chat room provides users with a real-time communication space, allowing interaction with other users while enjoying entertainment features.

- Key Features:
 - Instant Messaging: Users can freely send text messages, with all messages displayed in real-time in the chat window
 - Identity Differentiation: The system labels messages from the current user ("Me") differently from other users' messages through distinct colored bubbles
 - History Preservation: All chat content is saved locally and remains viewable when reopened
 - Streamlined Interface: Adopts familiar chat application layout with message list at top and input field at bottom
- Usage Scenarios:
 - Users can discuss queuing conditions, dishes, etc. while queuing
 - Users can discuss selection results with friends while using the wheel or dice features
 - Opportunity to share gaming tips or food recommendations
 - Seek hints or share scores during number guessing games
- Interaction Design:
 - Accessible via prominent green entry button at top
 - Semi-transparent overlay maintains visibility of main interface
 - Clear close button for easy exit from chat mode
 - Chronological message display with auto-scroll to newest messages

2. Number Guessing Game

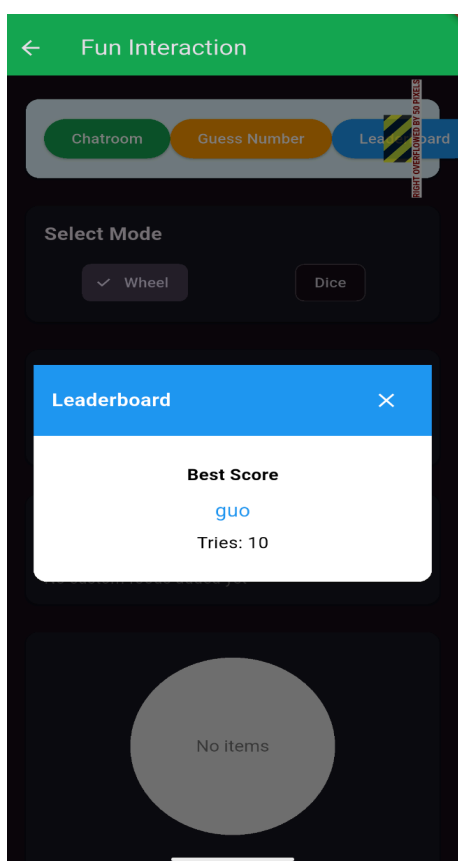
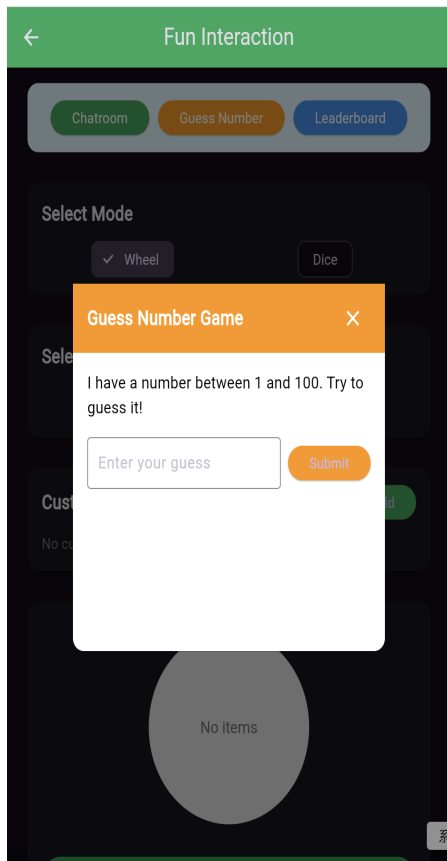
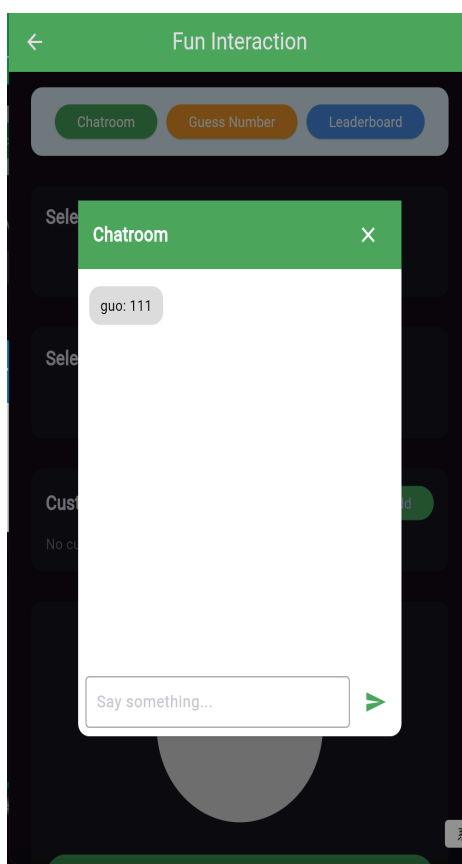
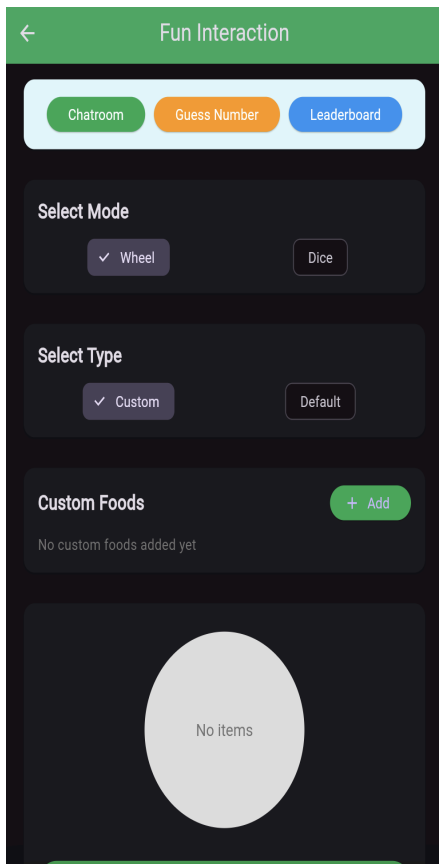
Game Mechanics: This classic number-guessing puzzle game randomly generates a number between 1-100 for users to identify through successive attempts.

- Game Flow:
 - System randomly selects a target number when game begins
 - User inputs and submits guessed number
 - System indicates if guess is "too high", "too low" or "correct"
 - User adjusts guesses based on feedback until correct answer is found
 - System records number of attempts and updates leaderboard
- Special Features:
 - Instant Feedback: Immediate hints after each guess maintain game rhythm
 - Attempt Counting: Automatic tracking of guesses encourages efficiency
 - Input Validation: Ensures only valid numbers are entered
 - Multilingual Hints: Displays feedback in Chinese or English based on system language

3. Leaderboard System

Design Purpose: Enhances user engagement through competition mechanics, motivating continuous improvement.

- Core Features:
 - Performance Recording: Saves users' best scores (fewest guesses) in number guessing game
 - Player Identification: Records record-setting player names (using username or default "Me")
 - Visual Presentation: Highlights best scores and achievers in card format
 - Empty State Handling: Displays encouraging prompts when no records exist
- User Experience:
 - Sense of Achievement: Strong accomplishment feeling when setting new records
 - Challenge Factor: Leaderboard motivates users to continually challenge themselves
 - Social Element: Potential for expansion to show friend rankings
- Interaction Design:
 - One-tap access via blue entry button
 - Clean data presentation avoids information overload
 - Visible close button for easy return to main interface
 - Responsive layout ensures good display across screen sizes



Vendor User features

1. Vendor Restaurant Management:

Implementation Methods

1. Automatic Matching of Restaurant ID and Name

Data Binding Logic:

- After logging in, the system retrieves the vendor's associated canteenId (e.g., canteenId_[username]) from SharedPreferences.
- The system matches the ID with a predefined list of restaurants (see _allCanteens in student_canteen_list.dart) to automatically display the corresponding restaurant name.

2. Operating Hours Management

Data Storage:

- Operating hours are stored in SharedPreferences under the key canteen_hours_[canteenId].
- Example format: 7:00 AM - 9:00 PM.

Editing Logic:

- Input time via TextFormField or a time picker (e.g., showTimePicker).
- Upon saving, update local storage and mark the restaurant as "updated" (triggering a red dot notification on the student side).

3. Interaction Flow

Vendor Logs In → The system automatically loads their bound restaurant information.

Modify Operating Hours:

- Enter new hours (e.g., 8:00 AM - 10:00 PM).
- Click the "Save" button to trigger the following actions:
 - Update operating hours in SharedPreferences.
 - Set canteen_unread_[canteenId] = true, showing a red dot notification on the student side.

Student-Side Sync:

- The next time students open the app, the restaurant card automatically updates the operating hours and displays a red dot.

6:14 5G

ByteBites - Vendor

vendor
@vendor

Nickname
vendor ✓

Canteen Name
Fine Food ✓

Description
great ✓

Opening Hours
✓

Address
✓

Canteen ID
8 ✓

Profile

6:13 5G

ByteBites - Vendor

Menu Management

Add Dish

Dish Name

Description

Price

Cancel Save

+

Menu

2. Reply to Students' Reviews:

Core Implementation

1. Data Storage and Synchronization

- All review data (including replies) is stored locally using SharedPreferences, with the key `canteen_reviews_${canteenId}`.
- Each review is represented as a Map, containing fields such as `id`, `comment`, `rating`, `reply`(vendor's reply content), `username`, `nickname`, etc.
- Vendors and students access and modify the same set of review data by sharing the same `canteenId`.

2. Reply Functionality Implementation

- Vendor Side (`vendor_reviews_page.dart`):
 - Clicking the reply icon (`Icons.reply`) on the right side of a review card opens a dialog (`AlertDialog`), allowing the vendor to input a reply.
 - Upon submission, `_saveReviews` is called to update the data, and `_addAnnouncement` generates a notification (e.g., "Vendor replied to your review").
- Student Side (`student_canteen_reviews_screen.dart`):
 - The reply field in the review data is checked to dynamically display the vendor's reply.

Demonstration of Effects

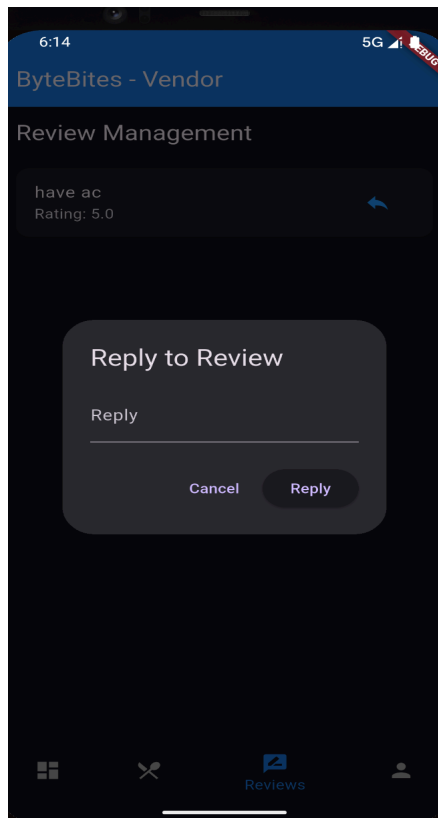
1. Vendor Workflow

- Navigate to the "Review Management" page → Click the reply icon on a review → Enter a reply and submit → The review card updates to show the reply.

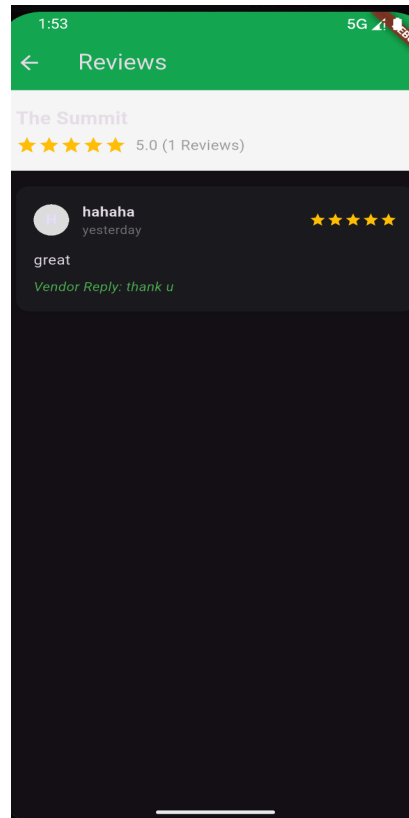
2. Student Workflow

- Navigate to the restaurant review page → View the vendor's reply in green text on the review card.

* reply from vendor



* in students' page



3. Dashboard Analysis

Key Features

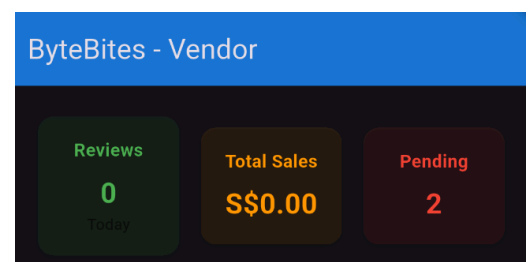
The vendor dashboard has been enriched with three main statistical components:

1. Reviews - Shows how many orders were placed today
2. Total Sales - Displays the total revenue generated
3. Pending - Indicates how many orders require attention

UX/UI Design Analysis

Visual Design

1. Card-Based Layout: Each statistic is presented in a card with:
 - Light background tint matching the category color
 - Bold title in the category color
 - Large, prominent value display
 - Optional subtitle for additional context
2. Color Coding:
 - Reviews (green) - suggests positive activity



- Total sales (orange) - indicates financial metrics
 - Pending orders (red) - draws attention to action items
3. Responsive Layout: Cards are arranged in a row with equal spacing, adapting to different screen sizes.

Information Architecture

1. Primary Metrics at the top for immediate visibility
2. Quick Access buttons below for common actions
3. Recent Reviews section at the bottom for context

Interaction Design

1. Data Loading: Shows a loading state while fetching data
2. Real-time Updates: Statistics update when new orders/reviews are added
3. Visual Hierarchy: Important numbers are large and bold

Functional Details

Reviews

- Calculated by counting reviews with today's date
- Helps vendors gauge daily activity
- Uses green color for positive connotation

Total Sales

- Sums up prices from all reviews (acting as order records)
- Formatted as currency (S\$)
- Shows two decimal places for precision
- Uses orange color to distinguish financial data

Pending Orders

- Counts reviews without replies
- Indicates items needing vendor attention
- Uses red color to highlight urgency

Technical Implementation Notes

1. Data Source: Uses SharedPreferences as a local database
2. Review Processing: Treats reviews as order records for simplicity
3. Date Handling: Parses ISO date strings to determine today's orders
4. State Management: Uses setState to update UI when data changes

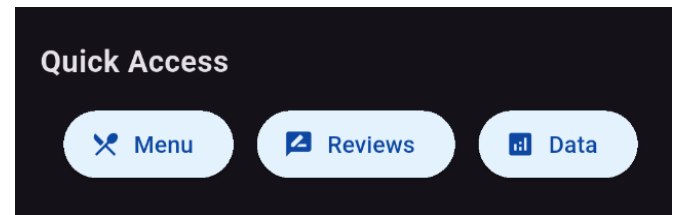
4. Quick Access

The vendor quick access feature provides vendors with convenient shortcuts to key sections of the app from the dashboard

Code Structure

The quick access section consists of:

1. A title ("Quick Access")
2. A horizontal row containing three buttons:
 - Menu button
 - Reviews button
 - Data button



Functionality

1. Menu Button

- Label: "Menu"
- Action: Switches to the menu management page (index 1 in the bottom navigation)
- Behavior:
 - Finds the parent `_HomeScreenVendorState`
 - Updates the `_selectedIndex` to 1
 - This shows the `VendorMenuPage` which allows vendors to:
 - Add/edit/delete dishes
 - Toggle dish availability
 - Reorder menu items
 - Upload dish images

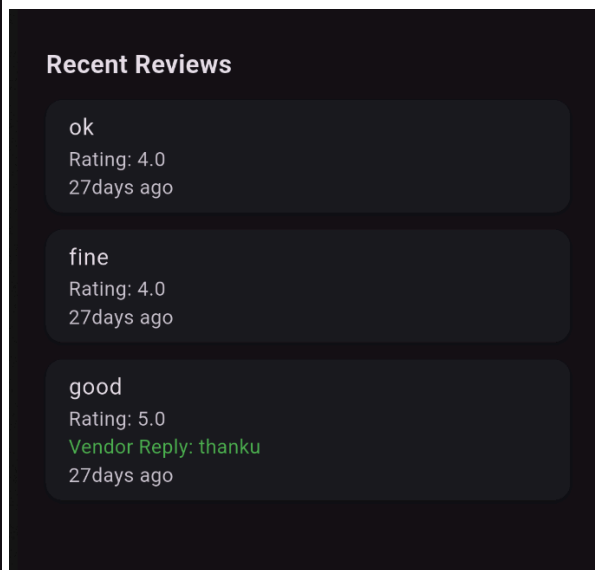
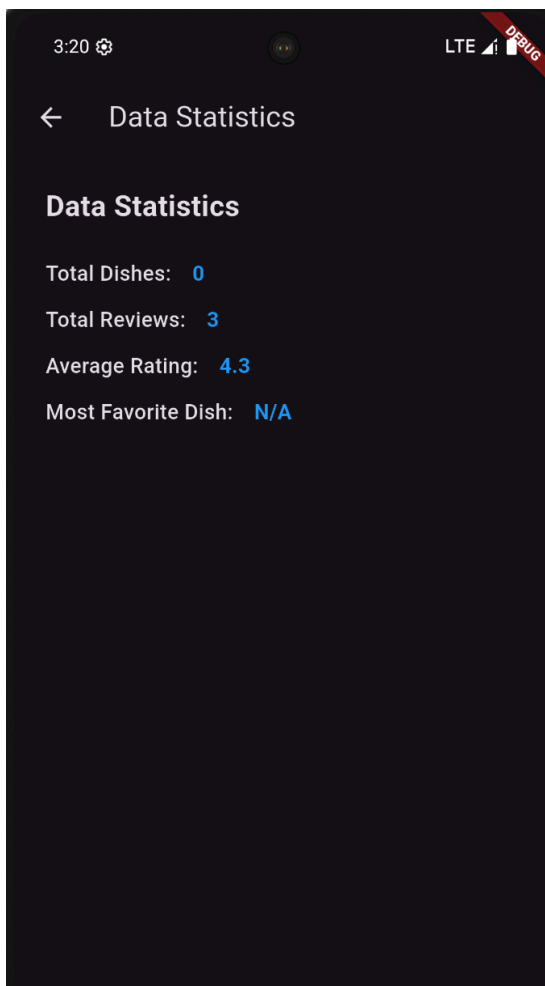
2. Reviews Button

- Label: "Reviews"
- Action: Switches to the reviews management page (index 2 in the bottom navigation)
- Behavior:
 - Finds the parent `_HomeScreenVendorState`
 - Updates the `_selectedIndex` to 2
 - This shows the `VendorReviewsPage` which allows vendors to:
 - View customer reviews
 - Filter to see only unreplied reviews
 - Add replies to reviews
 - See review statistics (average rating, total count)

3. Data Button

- Label: "Data"
- Action: Navigates to the `VendorDataPage` showing statistics

- Behavior:
 - Unlike the other buttons, this opens a new page rather than switching tabs
 - Shows key metrics including:
 - Total number of dishes
 - Total number of reviews
 - Average rating
 - Most popular dish
 - Data is fetched asynchronously when the page loads



5. Vendor Reviews

The vendor reviews system provides a comprehensive interface for canteen vendors to manage customer feedback, with features for replying to reviews, filtering, and highlighting unreplied messages.

Core Features

1. Review Management Dashboard

- Displays total review count and average rating
- Shows individual reviews with customer comments and ratings
- Visual indicators for unreplied reviews (yellow background)
- Reply functionality for each review

2. Filtering System

- Toggle to show only unreplied reviews
- Maintains original review indices for accurate replying

3. Data Visualization

- Summary statistics at the top (total reviews, average rating)
- Clear visual distinction between replied and unreplied reviews

UI/UX Design

1. Visual Hierarchy:

- Statistics prominently displayed at top
- Clear section headers
- Color-coded review cards (yellow for unreplied)

2. Interaction Design:

- Floating reply button on each review card
- Simple toggle for filtering unreplied reviews
- Clean dialog for composing replies

3. Feedback Mechanisms:

- Immediate visual feedback when replying
- Persistent storage of all changes
- Announcement system to notify users of vendor replies

4. Accessibility:

- Clear typography hierarchy
- Icon buttons with tooltips
- Color contrast for readability

Data Flow

1. Loading:

- Reviews loaded from SharedPreferences on init
- Parsed from JSON strings
- Statistics calculated immediately

2. Filtering:

- Toggle updates showOnlyUnreplied state
- Build method filters reviews based on current state

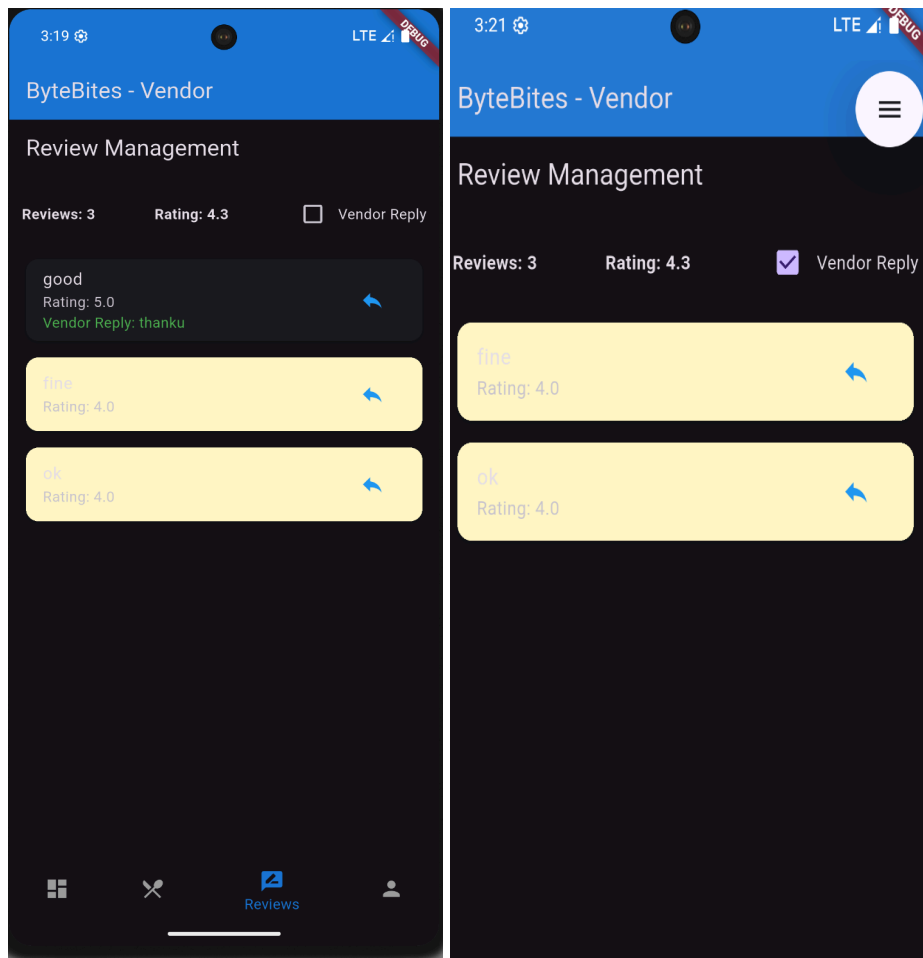
3. Replying:

- Reply text captured in dialog
- Original review updated in list
- Entire list saved back to SharedPreferences
-

Localization Support

Uses AppLocalizations for:

- "Only Unreplied" filter label
- Announcement messages
- UI text elements



6. Vendor Announcements

Overviews

The Vendor Announcements feature allows canteen vendors to publish announcement information, which is displayed as banner notifications at the top of the student interface. This serves as a two-way communication channel—vendors can post important notices (such as special offers, changes in business hours, etc.), and students can view these updates in real time.

UI/UX Design

Vendor-Side Announcement Interface

At the bottom of the vendor profile page (VendorProfilePage), there is an announcement posting section:

- Title: Displays bold text "Announcements"
- Input Field: A text input box with the placeholder "Enter announcement..."
- Send Button: A green send icon button on the right

- Announcement List: Published announcements are displayed as a bulleted list, each preceded by a dot symbol

Design Features:

- Simple and intuitive input interface
- New announcements are automatically added to the top of the list
- Real-time saving to ensure data persistence
- Synchronization with the global announcement system

Student-Side Announcement Display

The announcement bar is located below the AppBar on the student homepage:

- Height: 36 pixels
- Displays the latest announcement
- Designed as a banner to ensure visibility without disrupting the main content

Data Flow

Publishing Process:

1. Vendor enters announcement content in the input field
2. Clicking the send button triggers `_addAnnouncement`
3. The announcement is added to both the vendor-specific and global lists
4. The UI updates to display the new announcement

Display Process:

1. The student-side app loads global announcements upon launch
2. The `AnnouncementBar` component retrieves and displays the latest announcement
3. The component may include scrolling or carousel functionality to show multiple announcements

Technical Features

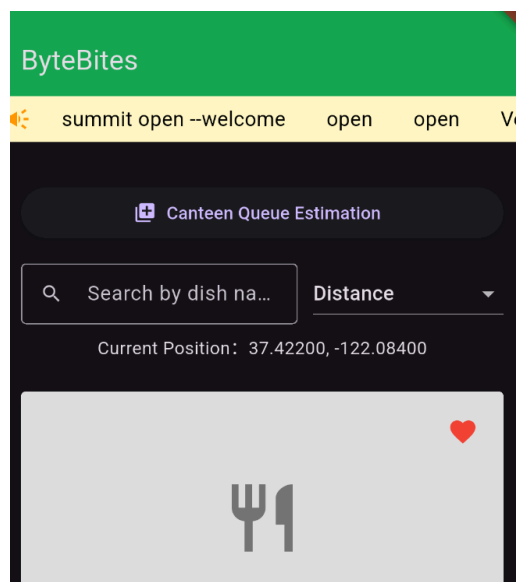
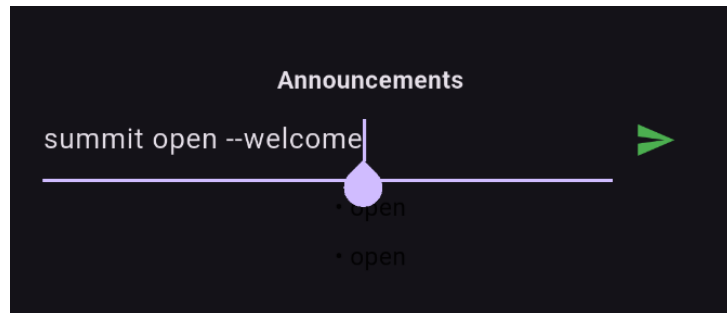
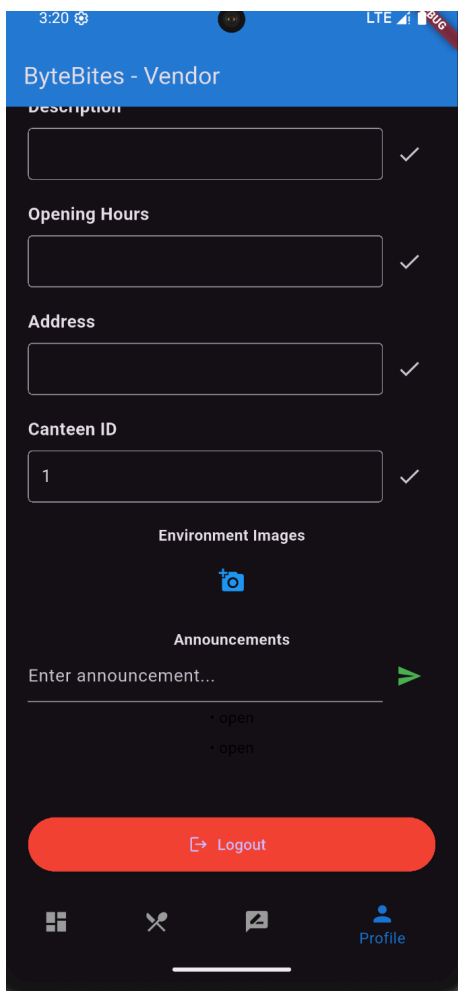
- Data Synchronization: Vendor announcements are stored in two places:
 - Vendor-specific list (for vendor management)
 - Global list (for student access)
- Data Structure: Global announcements use JSON format with the following fields:
 - `id`: Unique identifier (timestamp)
 - `message`: Announcement content
 - `canteenId`: ID of the canteen posting the announcement
 - `createdAt`: Creation time
- Persistence: Uses `SharedPreferences` for simple local storage, suitable for small-scale data

Potential Expansion Features

- Announcement Categorization: Distinguish between urgent notices, general updates, etc.
- Rich Text Support: Allow formatted text or images
- Expiration Date: Set automatic expiration for announcements
- Read/Unread Status: Track whether students have viewed announcements
- Multilingual Support: Provide announcements in different languages

Usage Scenarios

- Notifications about temporary canteen closures
- Special promotions or discounts
- Announcements for new menu items
- Changes in business hours
- Holiday-specific arrangements



Backend (Spring boot -> java)

1. Overview

The system adopts a frontend-backend separation architecture. The frontend is built using the React framework for user interfaces, while the backend provides RESTful API services using Spring Boot. Communication between the frontend and backend occurs via the HTTP protocol, with JSON as the data format. This architecture allows independent development, deployment, and scaling of the frontend and backend while ensuring seamless collaboration through well-defined interfaces.

2. Authentication and Authorization Integration

2.1 User Registration Process

The frontend sends a registration request containing a username, password, and email to the `/auth/register` endpoint. The backend first verifies the uniqueness of the username and email, then encrypts the password using the BCrypt algorithm before storing it. Upon successful registration, the backend generates a JWT token and returns it to the frontend, which stores it in `localStorage` or cookies for subsequent authentication.

2.2 User Login Process

The frontend sends the username and password to the `/auth/login` endpoint. The backend uses Spring Security's `AuthenticationManager` for authentication. If successful, it generates and returns a JWT token. The frontend includes this token in the `Authorization` header (formatted as `"Bearer <token>"`) for subsequent requests.

2.3 Token Management Mechanism

The JWT token has a validity period of 24 hours (86,400,000 milliseconds). The frontend handles token expiration by redirecting the user to the login page upon receiving a 401 Unauthorized response. Logout functionality is implemented by clearing the stored token on the frontend, with no token blacklist maintained on the backend.

3. Canteen Data Management Integration

3.1 Canteen List Display

The frontend retrieves all canteen data, including IDs, names, locations, images, and ratings, via a GET request to `/canteens`. To optimize performance, the frontend implements pagination and local caching to minimize unnecessary requests.

3.2 Canteen Details Display

When a specific canteen is selected, the frontend fetches detailed information (e.g., operating hours, description, coordinates) via GET `/canteens/{id}`. The map component uses these coordinates to pinpoint the canteen's location.

3.3 Canteen Management Functions

Admin users interact with canteen data via POST, PUT, and DELETE methods. All management requests require a valid JWT token, and the backend verifies user role permissions before executing operations. Form data is submitted in `FormData` format to support image uploads.

4. Dish Review System Integration

4.1 Review Display and Filtering

The frontend retrieves review data flexibly through multiple endpoints:

- GET `/dish-reviews/canteen/{canteenId}` fetches all reviews for a canteen.
- GET `/dish-reviews/canteen/{canteenId}/dish/{dishName}` retrieves reviews for a specific dish.
- GET `/dish-reviews/user/{username}` fetches a user's review history.

Reviews are displayed in descending order of creation time, with the frontend implementing infinite scroll for loading additional content.

4.1.1 Review Management System

The frontend handles user reviews through multiple access points:

1. POST `/reviews` submits new canteen reviews with rating and comment data.
2. GET `/dish-reviews/canteen/{canteenId}` retrieves all dish-specific reviews for a canteen.
3. GET `/dish-reviews/canteen/{canteenId}/dish/{dishName}` loads reviews filtered by specific menu items.

4.1.2 User-Centric Review Operations

The system provides personalized review access:

1. GET `/dish-reviews/user/{username}` fetches an individual user's entire review history across all canteens.

2. GET /dish-reviews/average-rating?canteenId=xxx&dishName=xxx calculates and returns average ratings for quality assessment.
3. GET /dish-reviews/distinct-dishes/{canteenId} identifies all dishes that have received reviews in a canteen.

4.1.3 Menu Management Functions

POST /menu-items allows authorized users to add new dishes to a canteen's menu with complete details.

4.1.4 User Profile System

The frontend manages user profiles through:

1. GET /user-profile/{userId} retrieves complete user profile information including preferences.
2. PUT /user-profile/{userId} updates profile elements like nickname, avatar, dietary preferences and language settings.

4.2 Review Statistics Functionality

The frontend uses GET /dish-reviews/average-rating to fetch average dish ratings for star displays and sorting. GET /dish-reviews/distinct-dishes/{canteenId} retrieves a list of reviewed dish names to generate a "Popular Dishes" filter.

4.3 Review Submission and Management

When submitting a review, the frontend collects the dish name, rating (1-5 stars), and comment, then sends it via POST /dish-reviews. Editing and deletion are handled via PUT and DELETE methods, with the backend verifying that the requester is the review author.

5. Menu Management System Integration

5.1 Menu Display

The frontend retrieves canteen menu data via GET /menu-items/canteen/{canteenId}, grouping items by category. To optimize loading, the frontend initially loads only category lists, fetching specific dishes upon category selection.

5.2 Menu Management

Authenticated canteen owners can manage menu items:

- Add dishes: POST /menu-items, including dish name, price, description, and image.
- Modify dishes: PUT /menu-items/{id}, updating dish information.
- Delete dishes: DELETE /menu-items/{id}.

All operations include permission checks to ensure only canteen owners can modify their menus.

6. User Profile System Integration

6.1 Profile Retrieval and Display

The frontend fetches user profiles via GET /user-profile/{userId}, including nickname, avatar, dietary preferences (e.g., vegetarian, gluten-free), and language preferences. This data personalizes the UI and content recommendations.

6.2 Profile Editing

Users update their profiles via PUT /user-profile/{userId}, employing a progressive save strategy:

1. Avatar uploads use a separate endpoint, returning a URL for user object updates.
2. Basic information is submitted via form.
3. Preference changes are saved in real-time to minimize user steps.

7. Frontend-Backend Communication Details

7.1 Request and Response Formats

All APIs follow RESTful conventions:

- Success: 200 status code + JSON data.
- Errors: 4xx/5xx status code + error message JSON.
- Paginated data: Includes metadata (total count, page number) and data arrays.

7.2 Performance Optimization

- Data caching: Frequently accessed data (e.g., canteen lists) is cached.
- Request batching: Multiple data requests are sent in parallel via Promise.all.
- Lazy loading: Non-critical data is loaded on demand.
- Compression: Gzip reduces payload size.

8. Security Measures

8.1 Authentication Security

- JWT uses HS512 algorithm with a sufficiently long secret key.

- Password transmission is encrypted via HTTPS.
- Password storage uses BCrypt hashing with salt.

8.2 Data Security

- All modification operations validate user permissions.
- Users can only modify their own data and content.
- Sensitive fields (e.g., passwords) are never returned to the frontend.

8.3 Input Validation

- Frontend: Basic checks (required fields, formats).
- Backend: Business rule and data relationship validation.
- XSS prevention: Output encoding of special characters.
- SQL injection prevention: Parameterized queries.

9. Cross-Platform Compatibility

9.1 Responsive Design

The frontend adjusts layouts based on device screen size, ensuring optimal experiences on mobile, tablet, and desktop.

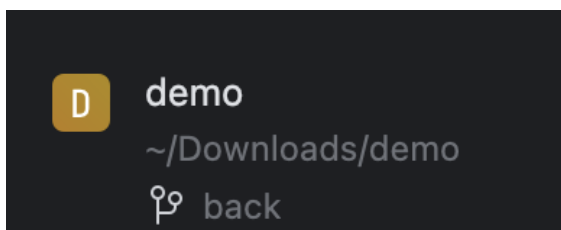
9.2 Cross-Origin Support

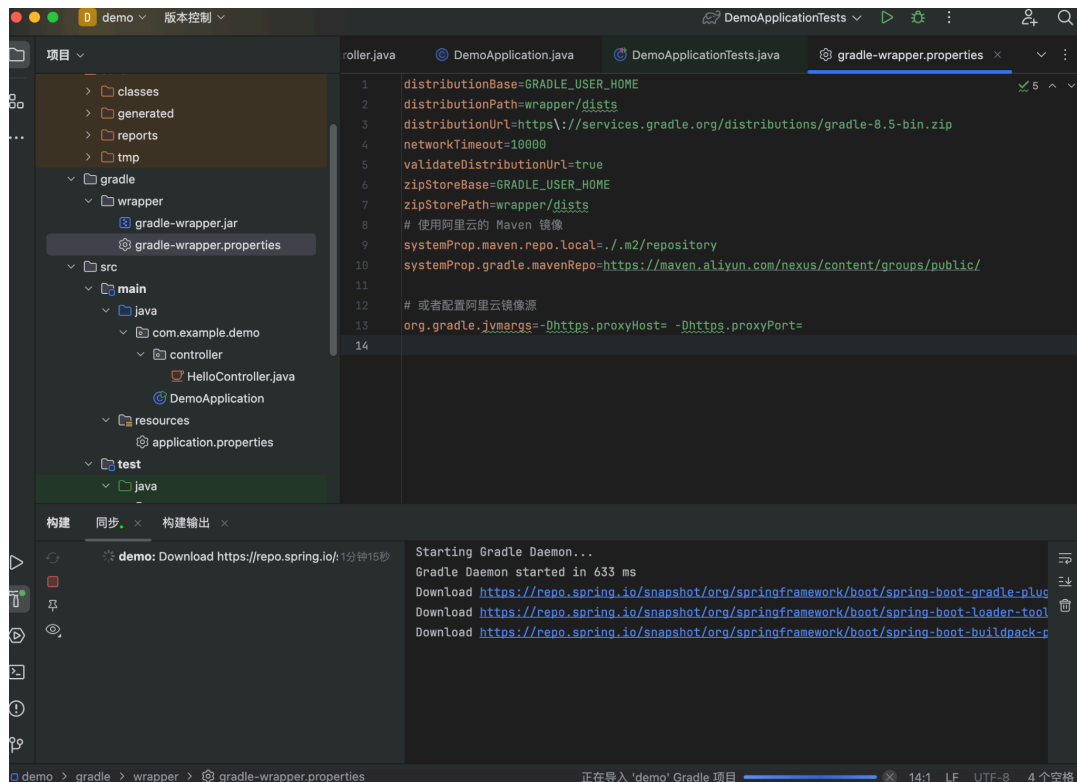
The backend configures CORS to allow requests from specified origins (e.g., localhost in development, production domains in live environments).

9.3 Multilingual Support

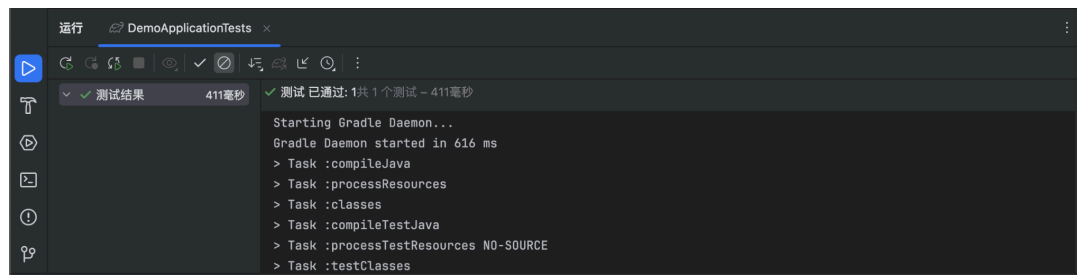
User language preferences are stored in profiles, with the frontend loading corresponding language resource files.

10. Configuration





The JUnit test task was carried out and the **BUILD SUCCESSFUL**



Through these multi-layered integrations, the frontend and backend form a complete, efficient, and secure campus canteen review system, delivering a seamless experience for browsing and sharing dining information.

Team Coordination & Quality Assurance

1. UI Consistency Check
 - Verified color scheme and style uniformity across interfaces
2. Key Achievements
 - Established complete development environment.
 - Implemented a functional authentication system.
 - Created differentiated interfaces for students and vendors.

- Developed core canteen information display.
- Laid foundation for advanced features (multilingual, avatar)

3. Challenges Encountered

- Initial Flutter environment configuration complexities.
- Time-consuming implementation of password-matching logic.
- Difficulty in coordinating interface styles between team members
- Technical hurdles in implementing album connection for avatars

Testing Process

Introduction

1. Purpose of Testing

This test plan aims to validate the completeness and reliability of the vendor reply feature, ensuring the following core requirements are met:

- Vendor Side: Ability to correctly submit, save, and display replies to reviews
- Student Side: Real-time visibility of vendor replies
- Data Consistency: Reply content persists after app restart
- Error Handling: Effective prevention of invalid data submissions (e.g., empty replies)

2. Scope of Testing

Covers the following key modules:

- Functional Logic: Reply submission, storage, and display (vendor_reviews_page.dart)
- Data Synchronization: Shared data between vendors and students via SharedPreferences
- UI Interaction: Reply dialog, review card styling (green italic text for replies)
- Notification System: Automatic notification generation after replies

3. Expected Output

- All test cases pass, with functionality meeting requirements
- Test report generated, detailing passed/failed cases and defects

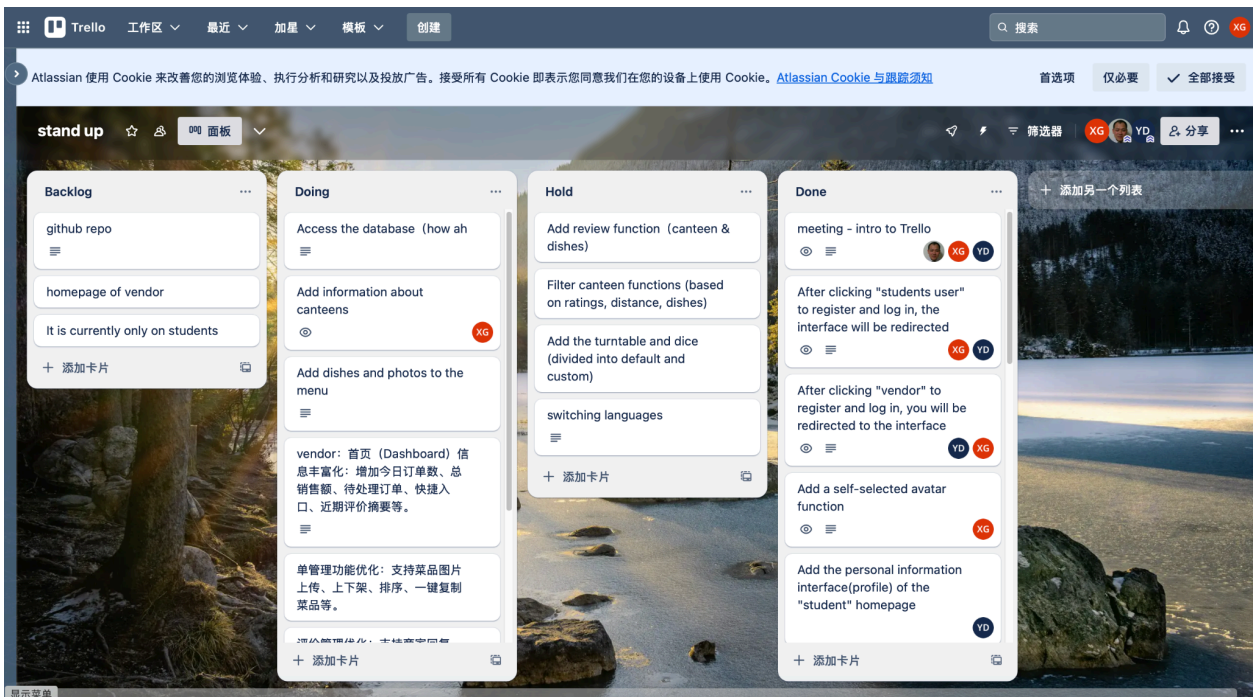
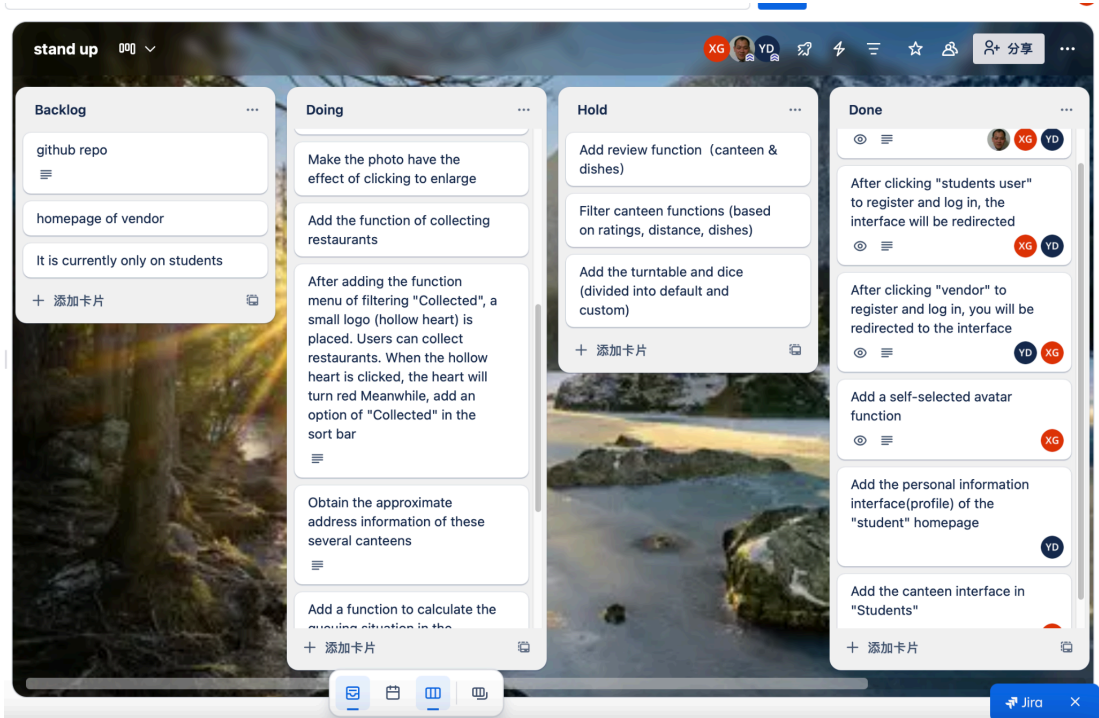
Test ID	User Story	Testing Objective	Steps to Execute	Expected Result	Pass/Fail
001	As a vendor, I want to reply to student reviews so I can address their feedback	Verify vendor can successfully reply to a review	1. Open vendor review page 2. Click reply icon on a review 3. Enter reply text	1. Reply dialog appears 2. Reply is saved 3. Review card updates with reply	pass

			4. Submit reply	4. Announcement is created	
002	As a student, I want to see vendor replies to my reviews	Verify replies are visible to students	1. Vendor submits reply (001) 2. Open student review page	Reply displays below original review in green italic text	pass
003	As a system, I want to notify students of new replies	Verify announcement system works	1. Vendor submits reply 2. Check SharedPreferences	New announcement exists in announcements list with correct message and unread flag	pass
004	Edge Case: Empty reply	Prevent empty replies from being saved	1. Open reply dialog 2. Attempt to submit empty reply	1. Dialog remains open 2. No data saved 3. Error displayed (if implemented)	pass
005	Data Persistence	Verify replies persist after restart	1. Submit reply (001) 2. Restart app 3. Check reviews	Reply remains visible in both vendor and student views	pass
006	UI Verification	Confirm proper UI rendering	1. Load pages with/without replies	1. Reply icon only shows on vendor side 2. Reply styling matches spec (green/italic) 3. Dialog has correct fields	pass
007	Multi-review Handling	Verify system works with multiple reviews	1. Create multiple reviews 2. Add replies to some 3. Check display	Only reviews with replies show vendor response	pass

Use of Trello

Using Trello to assign and track tasks helps make the development process more transparent and organized. Through a visual Kanban-style board, team members can clearly see the status of each task (e.g., To Do, In Progress, Done), along with assignees and deadlines. This not only improves collaboration and communication within the team, but also makes the project's progress easy to monitor, reducing the chances of duplicated work or missed tasks. Additionally, mentors or evaluators can

easily review the overall planning and execution of the project, improving management efficiency.



*main branch

main

3 Branches

0 Tags

Go to file

t

+

<> Code

 Xingchen722

feat: integrate backend canteen list api for student page

7e84111 · 4 days ago

🕒 2 Commits

📁 .gradle	feat: sync local changes and optimizations	4 days ago
📁 android	feat: sync local changes and optimizations	4 days ago
📁 assets/images	feat: sync local changes and optimizations	4 days ago
📁 ios	feat: integrate backend canteen list api for student ...	4 days ago
📁 lib	feat: integrate backend canteen list api for student ...	4 days ago
📁 linux	feat: sync local changes and optimizations	4 days ago
📁 macos	feat: sync local changes and optimizations	4 days ago
📁 test	feat: sync local changes and optimizations	4 days ago
📁 web	feat: integrate backend canteen list api for student ...	4 days ago
📁 windows	feat: sync local changes and optimizations	4 days ago
📄 .gitignore	feat: sync local changes and optimizations	4 days ago
📄 .metadata	feat: sync local changes and optimizations	4 days ago
📄 README.md	feat: sync local changes and optimizations	4 days ago
📄 analysis_options.yaml	feat: sync local changes and optimizations	4 days ago
📄 devtools_options.yaml	feat: sync local changes and optimizations	4 days ago
📄 l10n.yaml	feat: sync local changes and optimizations	4 days ago

*Backend

back

3 Branches

0 Tags

This branch is 5 commits ahead of, 2 commits behind main.

Xingchen722

feat: 菜品评价 (DishReview) 和用户资料接口补全, 支持前端联调

e8e9c08 · 4 days ago

Software Engineering Practices

To ensure robust and maintainable development, we will adopt:

1. Agile Development

- Have biweekly sprints with clear goals and tasks.
- Hold daily standups to discuss progress and issues.

2. Version Control (Git)

- Commit code frequently with meaningful commit messages.
- Use branches to manage different features.

3. Code Review

- Conduct regular code reviews to maintain code quality.
- Use consistent coding standards.

4. Testing

- Write unit and integration tests to ensure code correctness.
- Use automated testing tools like Cypress for UI testing.

5. Documentation

- Write API and project documentation to ensure team understanding of requirements and features.

6. Continuous Integration/Continuous Deployment (CI/CD)

- Automate testing and deployment processes to ensure code quality and fast releases.

7. Modular Code Structure

- Break code into independent modules for easier maintenance and scalability.

8. Monitoring and Logging

- Use error tracking tools and logging to resolve issues quickly.

9. Security

Validate user inputs and use encryption to protect sensitive data.

10. Performance Optimization

- Optimize performance bottlenecks and implement caching and lazy loading.

11. Team Collaboration

- Use tools (such as Trello and Jira) for task management and maintain smooth communication within the team.

Tech Stack

UI/UX: Figma

- Design each screen (home, profile, settings, etc.)
- Create a user flow (e.g., login → home → view details → back)
- Get feedback and iterate on the design

Cross-Platform Framework (Frontend): Flutter (Dart)

Backend & Database: Spring boot & PostgreSQL

- Storing authentication information safely
- Storing user data
- Restaurant stores and canteen information

Cloud Deployment: flutter+Dart (frontend) +Spring boot (backend)

Future improvements

1. Backend Database development

To improve data storage and accessibility, we plan to integrate PostgreSQL as our primary relational database. Key enhancements include:

- **Normalization of Data:** Designing efficient relational schemas for users, canteens, stalls, dishes, and queue logs to reduce redundancy and improve query performance.
- **API Integration:** Building secure endpoints using Spring Boot to allow frontend apps to interact with PostgreSQL seamlessly.
- **Authentication Layer:** Implementing role-based access control (e.g., admin, user) using JWT for secure operations like queue updates or stall management.
- **Performance Optimization:** Indexing commonly queried fields (e.g., stall IDs, timestamps) and using caching mechanisms (e.g., Redis) for frequently accessed data like canteen information.

2. UI/UX design improvement

We aim to improve the user experience and attractiveness of the app:

- **Consistent Visual Language:** Enhancing the design system (colours, icons, spacing) for clarity and modern appeal, using tools like Figma for prototyping.
- **Accessibility Support:** Introducing features like font scaling, dark mode, and screen reader compatibility to accommodate diverse user needs.
- **Interactions:** Adding animations (e.g., loading spinners, transition effects) to make the experience more responsive and intuitive.

3. Model Training for Queue Length Prediction

We aim to develop a machine learning (ML) model that predicts queue lengths at different stalls and estimates waiting times based on historical data:

- **Data Collection:** Gather time-stamped queue data, weather info, day of the week, and event schedules (e.g., lunch hour, exam season).
- **Model Development:** Train regression models (e.g., Linear Regression, Random Forest) or time-series models (e.g., LSTM) to predict wait times.
- **Evaluate performance** using MAE/RMSE metrics.

Remote testing

<https://xingchen722.github.io/Orbital1/>

(It might load slowly and you need to wait for a short while)

* generate gh-pages

gh-pages

5 Branches

0 Tags

Go to file

+

Code

Xingchen722

fix: set base href for GitHub Pages

✓

fada14a · 4 days ago

1 Commit

assets	fix: set base href for GitHub Pages	4 days ago
canvaskit	fix: set base href for GitHub Pages	4 days ago
icons	fix: set base href for GitHub Pages	4 days ago
.last_build_id	fix: set base href for GitHub Pages	4 days ago
favicon.png	fix: set base href for GitHub Pages	4 days ago
flutter.js	fix: set base href for GitHub Pages	4 days ago
flutter_bootstrap.js	fix: set base href for GitHub Pages	4 days ago
flutter_service_worker.js	fix: set base href for GitHub Pages	4 days ago
index.html	fix: set base href for GitHub Pages	4 days ago
main.dart.js	fix: set base href for GitHub Pages	4 days ago
manifest.json	fix: set base href for GitHub Pages	4 days ago
version.json	fix: set base href for GitHub Pages	4 days ago

About

No description, website, or topics provided.

Activity

0 stars

1 watching

1 fork

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Deployments 17

github-pages 4 days ago

+ 16 deployments

Languages

GitHub Pages

[GitHub Pages](#) is designed to host your personal, organization, or project pages from a GitHub repository.

Your site is live at <https://xingchen722.github.io/Orbital1/>
Last [deployed](#) by [Xingchen722](#) 4 days ago

Visit site



Build and deployment

Source

Deploy from a branch

Branch

Your GitHub Pages site is currently being built from the gh-pages branch. [Learn more about configuring the publishing source for your site.](#)

gh-pages

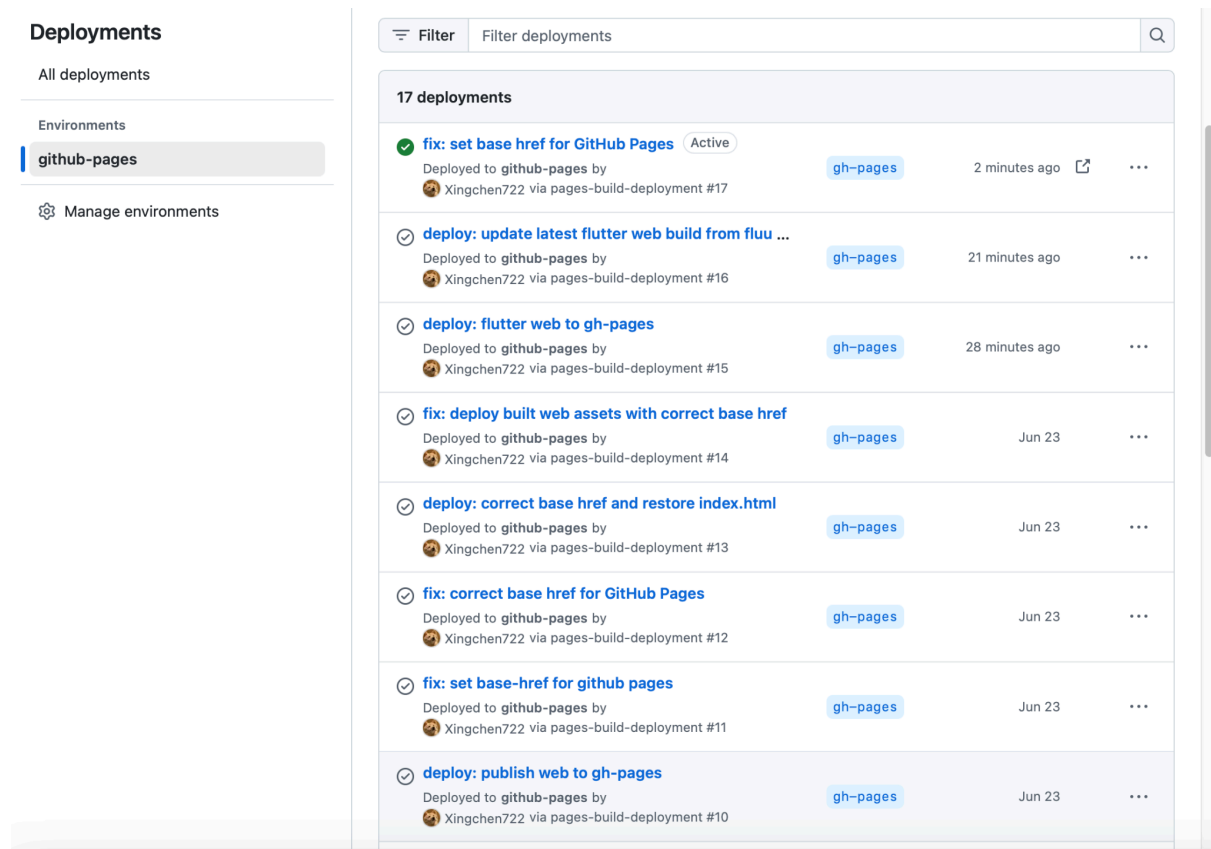
/ (root)

Save

Learn how to [add a Jekyll theme](#) to your site.

Your site was last deployed to the [github-pages](#) environment by the [pages build and deployment](#) workflow. [Learn more about deploying to GitHub Pages using custom workflows](#)

*update and get link



Create a GitHub Pages Site and Deploy a Web Test Version

1. Prepare the Web Files

- Ensure there is a static web version (usually index.html plus related CSS and JS files).
- For frontend frameworks (React, Vue, Flutter Web, etc.), build the static files first (e.g., npm run build for React, flutter build web for Flutter).

2. Create a GitHub Repository

- Log into GitHub and create a new public repository.
- Upload the web files (including index.html) to the repository's main branch (main or master).

3. Enable GitHub Pages

- Go to Settings in the repository.
- Find the Pages section.
- Under Source, select the branch (main or master) and folder (usually /root if files are in the root directory).
- Click Save.
- Wait a few minutes; GitHub will generate a URL

